

ANTHROPIC CERTIFICATION · EXAM CCDV-F

CCDV-F

Claude Certified Developer — Foundations

95 PRACTICE QUESTIONS · FULL ANSWER KEY WITH EXPLANATIONS · VERSION 3.0

95

TOTAL QUESTIONS

1

CORRECT ANSWER EACH

100%

EXPLAINED

SECTION 1

Questions 1–19

Claude Certified Developer — Foundations · Practice Set

Q001

SCENARIO · SINGLE CHOICE

Your Claude application's API keys are stored in a secrets manager. The team is debating whether the same key should be used in development, staging, and production environments. How would you handle the keys across environments?

✓ **A. Use distinct keys for each environment so a compromise in one environment does not affect the others during normal operation across the application's lifecycle.**

B. Rotate the same key across environments at random intervals on the grounds that random rotation provides isolation between environments without requiring distinct keys.

C. Use the same key across all environments for simplicity and treat the propagation of any compromise as a known operational tradeoff for the team's key management approach.

D. Use a single development key everywhere on the grounds that production keys are too risky to deploy across the application's three environments during normal operation.

✓ **Correct Answer: A. Use distinct keys for each environment so a compromise in one environment does not affect the others during normal operation across the application's lifecycle.**

Why: Distinct per-environment keys contain the blast radius of a compromise to a single environment. A shared key (C) couples the failure domains together, and random rotation (B) or using a dev key everywhere (D) doesn't provide real isolation.

Q002

SCENARIO · SINGLE CHOICE

Your Claude application returns confident-sounding answers, but occasionally those answers contain factual errors that downstream systems treat as ground truth. The team is concerned about the application's confidence-versus-accuracy gap. How would you address the gap?

A. Lower the model's temperature so the model's responses sound less confident and downstream systems are less likely to treat the responses as ground truth in normal operation.

✓ **B. Apply skepticism toward confident output by adding validation steps, sourcing requirements, or confidence calibration before treating outputs as ground truth.**

C. Reject every response the application produces until a manual accuracy review is conducted on each response by

a human reviewer before any downstream system uses it.

D. Add a disclaimer to every output telling users to verify the accuracy of the output and treat the disclaimer as the primary mechanism for managing the confidence-versus- accuracy gap.

✓ **Correct Answer: B. Apply skepticism toward confident output by adding validation steps, sourcing requirements, or confidence calibration before treating outputs as ground truth.**

Why: Fluent, confident phrasing is not a reliability signal. The right response is to add validation, sourcing, or confidence calibration so downstream systems don't treat generated text as verified fact. Lowering temperature (A) or adding a disclaimer (D) doesn't change whether the content is actually correct.

Q003

SCENARIO • SINGLE CHOICE

A teammate has asked you to explain the difference between context engineering and prompt engineering. They have heard the terms used interchangeably and are unsure how each applies to a Claude application that processes long-running multi-step tasks. How would you describe the distinction?

A. Prompt engineering focuses on the model's response, while context engineering focuses on the user's input across many sessions in a long-running multi-step Claude application.

B. Prompt engineering is the older term for prompt design, while context engineering is the newer term that has replaced it in modern Claude applications across the industry.

✓ **C. Prompt engineering shapes individual prompts for specific outputs, while context engineering manages how content flows across turns and steps and takes steps to keep relevant state visible.**

D. Prompt engineering and context engineering each address content the team gives Claude, but the team can group them under a single workflow because the practices use overlapping techniques.

✓ **Correct Answer: C. Prompt engineering shapes individual prompts for specific outputs, while context engineering manages how content flows across turns and steps and takes steps to keep relevant state visible.**

Why: Prompt engineering optimizes what you ask for in a single turn; context engineering manages what information stays visible and relevant across an entire multi-step, multi-turn task. They're related but solve different problems, so treating them as interchangeable (B, D) or splitting them along input/output lines (A) misses the distinction.

Q004

SCENARIO • SINGLE CHOICE

You are choosing between using STDIO-based communication and HTTP-based communication for an MCP server. The server will be invoked by a Claude Code session running locally. Which communication pattern would you use?

A. STDIO and HTTP simultaneously for redundancy, with the team running two communication patterns at the same time across Claude Code sessions.

B. HTTP on the grounds that HTTP is preferable to STDIO in any context the application might encounter during normal operation.

C. HTTP polling against the MCP server, treating HTTP as a substitute for the standard MCP communication patterns.

✓ **D. STDIO because STDIO is well-suited to local process communication and avoids unnecessary network setup.**

✓ **Correct Answer: D. STDIO because STDIO is well-suited to local process communication and avoids unnecessary network setup.**

Why: STDIO is the natural fit for a local process talking to a local Claude Code session — no network stack, no ports, minimal setup. HTTP (B, C) is meant for remote or multi-client scenarios, and running both at once (A) adds needless complexity.

Q005

SCENARIO • SINGLE CHOICE

A teammate has asked why your Claude application sometimes produces a response that includes the prompt text repeated back, and other times produces a response with the prompt text rephrased. They suspect a bug in the application's request construction. How would you respond?

A. Tell the teammate that the variation depends on which Claude model serves the request, and recommend pinning the application to a single model version to make the output behavior consistent.

✓ **B. Explain that LLMs generate output token by token, and variation in how prompt content appears in output is a property of generation, not a bug in request construction.**

C. Confirm that the variation is a bug in the application's request construction and start investigating which part of the application is producing the inconsistent prompt text.

D. Tell the teammate that the variation is caused by the application sending two different prompts on different runs and propose a code change that pins the system prompt to a single version.

✓ **Correct Answer: B. Explain that LLMs generate output token by token, and variation in how prompt content appears in output is a property of generation, not a bug in request construction.**

Why: LLMs generate text token by token from a probability distribution, so surface-level variation in how prompt content reappears in output is expected generation behavior, not evidence of a bug in request construction (C) or of multiple prompt versions being sent (D). Pinning the model (A) wouldn't change this fundamental behavior.

Q006

SCENARIO • SINGLE CHOICE

Your team is integrating Claude into an existing REST API service. The service handles concurrent requests, and you are deciding how to structure the Claude API calls within the existing async codebase. How would you structure the Claude calls?

A. Run Claude API calls in a separate thread pool isolated from the rest of the service so the main event loop is not affected.

B. Replace the existing async REST service with a synchronous service to match the call style of synchronous Claude SDK clients.

✓ **C. Use the Claude SDK's async client and structure Claude calls as awaitable operations alongside the rest of the service's async code.**

D. Make synchronous Claude API calls and block the async event loop until each call completes before processing the next request.

✓ **Correct Answer: C. Use the Claude SDK's async client and structure Claude calls as awaitable operations alongside the rest of the service's async code.**

Why: The Claude SDK's async client lets Claude calls sit naturally alongside the rest of an async REST service's event loop. Isolating calls in a separate thread pool (A) or making blocking synchronous calls (D) fights the existing async architecture instead of working with it, and rewriting the whole service as synchronous (B) is a disproportionate change.

Q007

SCENARIO · SINGLE CHOICE

Your Claude application's content policy specifies categories of content it should not produce under any circumstance. The application currently has no mechanism to enforce this policy, and content matching these categories is appearing in the application's output. How would you enforce the content policy?

A. Enhance the system prompt to contain explicit instructions for the categories to avoid, complete with examples of each category. Treat the strengthened prompt as the primary enforcement mechanism for the application's content policy across all responses.

B. Remove the content policy entirely and let any output reach users during normal operation, accepting whatever content the application produces in response to incoming traffic.

C. Move enforcement to users by asking them to report content policy violations after the violating content has already reached them in the application's responses.

✓ **D. Add deterministic output filtering that checks responses against the content policy before they reach users.**

✓ **Correct Answer: D. Add deterministic output filtering that checks responses against the content policy before they reach users.**

Why: A hard content policy needs a deterministic, code-level check that runs on every response before it reaches users — prompting alone (A) is probabilistic and can be bypassed, and pushing enforcement to users after the fact (C) or dropping the policy (B) doesn't prevent the harm.

Q008

SCENARIO · SINGLE CHOICE

You are implementing a custom tool for your Claude agent. The tool needs to interact with an external pricing service that returns product data. Which of the following best practices would you apply as you develop this tool?

A. Omit the tool description and let the model infer when to use the tool based on the tool's name and the rest of the prompt context.

B. Define the tool with a loose schema and let the model interpret the inputs flexibly on each call the agent makes.

C. Implement the tool with no error handling and let the agent loop catch failures whenever the pricing service returns an error during operation.

✓ **D. Define the tool with a clear schema, write a precise description for when to call it, and handle pricing service errors explicitly.**

✓ **Correct Answer: D. Define the tool with a clear schema, write a precise description for when to call it, and handle pricing service errors explicitly.**

Why: Good tool design combines a clear schema, a precise description of when to call the tool, and explicit handling of the external service's failure modes. Skipping the description (A), loosening the schema (B), or ignoring errors (C) all push ambiguity or fragility onto the agent loop at runtime.

Q009

SCENARIO · SINGLE CHOICE

You are setting up a CI/CD pipeline for a new Claude application. The pipeline needs to run automated checks on every pull request before code can be merged. The CI/CD checks would include...

✓ **A. Automated tests of the Claude integration, linting, and any other standard quality gates the team applies to its other services.**

B. A full end-to-end production deployment on every pull request to catch all possible issues before any code is merged into the main branch.

C. Automated tests of the Claude integration only, with linting handled separately during local development on each developer's machine.

D. Automated linting and security scanning, with Claude integration testing handled manually during pre-release verification by a designated reviewer.

✓ **Correct Answer: A. Automated tests of the Claude integration, linting, and any other standard quality gates the team applies to its other services.**

Why: Claude-powered code should go through the same quality gates as any other service code — tests, linting, and standard checks on every PR. Full production deploys on every PR (B) is excessive, and splitting checks across manual and automated steps (C, D) leaves gaps before merge.

Q010

SCENARIO · SINGLE CHOICE

A new Claude model release includes performance improvements for several reasoning tasks but has changed the format of its responses to system prompts that use multi-section instructions. Your application uses multi-section system prompts heavily. Initial evaluation on the application's actual workload shows the new model performs 8 percent better on reasoning tasks but produces malformed output on roughly 3 percent of requests because of the format change. The team is debating whether to upgrade. How would you decide?

A. Upgrade immediately, because the 8 percent reasoning improvement outweighs the 3 percent malformed output rate across the application's typical request distribution.

✓ **B. Adapt the application's system prompt to the new model's format expectations and re-evaluate, then upgrade only if the adapted prompt eliminates the malformed output while preserving the reasoning improvements.**

C. Upgrade and add a downstream validation step that catches the 3 percent malformed output before it reaches users, treating the validation step as the team's mitigation for the format change.

D. Stay on the previous model permanently to avoid the malformed output rate and any future format changes that subsequent model releases might introduce in the application.

✓ **Correct Answer: B. Adapt the application's system prompt to the new model's format expectations and re-evaluate, then upgrade only if the adapted prompt eliminates the malformed output while preserving the reasoning improvements.**

Why: Because the new model changed output formatting for a feature the application depends on heavily, the responsible path is to adapt the system prompt to the new format, re-evaluate, and only upgrade once the malformed-output issue is actually resolved — not ship a known regression (A, C) or freeze on the old model indefinitely (D).

Q011

SCENARIO • SINGLE CHOICE

A teammate is reviewing the team's threat model for a Claude application and has asked you to identify the categories of AI-specific threats that the model should cover. The teammate has already listed traditional web application threats and wants to know what additional categories apply to a Claude application. Which AI-specific threat categories would you add?

A. Cross-site scripting and SQL injection, because these traditional web application threats apply with equal weight to any application that uses Claude in any way.

B. Network-level denial of service and physical infrastructure attacks, because these categories cover the threats most likely to affect any Claude application in production.

✓ **C. Prompt injection, data leakage from prompts or context, jailbreak attempts, and unsafe model output that bypasses application controls.**

D. Supply chain attacks on the Claude SDK because the SDK itself is the only point of vulnerability that a Claude application introduces beyond traditional web application threats.

✓ **Correct Answer: C. Prompt injection, data leakage from prompts or context, jailbreak attempts, and unsafe model output that bypasses application controls.**

Why: Prompt injection, data leakage through prompts/context, jailbreaks, and unsafe outputs that slip past application controls are threats that are specific to LLM-powered applications and don't show up in a traditional web app threat model. Traditional web threats (A) and generic infrastructure threats (B, D) don't capture what's unique about a Claude application.

Q012

SCENARIO • SINGLE CHOICE

Your Claude agent's hooks are currently triggered for every action, which slows down the agent significantly even when actions pose no risk. The team wants to scope hooks more carefully. How would you scope the hooks?

✓ **A. Scope hooks to only the high-risk actions, such as destructive operations or sensitive data access, and remove hooks from low-risk actions to balance safety with performance.**

- B. Disable all hooks while the team re-scopes them, treating the period of no hook enforcement as a temporary state during the re-scoping work.
- C. Disable the agent during peak hours so the hook overhead does not slow the application down during the busiest periods of the day across the application's operation.
- D. Replace hooks with system prompt instructions on the grounds that prompt instructions can produce the same enforcement effect that hooks produce on the agent's actions.

✓ **Correct Answer: A. Scope hooks to only the high-risk actions, such as destructive operations or sensitive data access, and remove hooks from low-risk actions to balance safety with performance.**

Why: Scoping hooks to high-risk, destructive, or sensitive actions keeps enforcement where it matters while removing overhead from low-risk actions. Disabling hooks (B), disabling the agent (C), or replacing them with prompt instructions (D) all trade away the deterministic safety hooks provide.

Q013

SCENARIO · SINGLE CHOICE

Your agent is processing tasks that take 30 to 60 minutes to complete. Each task has well-defined intermediate checkpoints, and the team wants the agent to be able to resume from the most recent checkpoint if a process is interrupted. How would you implement this resumability?

- A. Increase the agent's timeout to several hours so that interruptions become rare enough to ignore in practice.
- ✓ **B. Apply a checkpointing pattern that persists the agent's intermediate state and reloads that state when resuming an interrupted task.**
- C. Run two copies of the agent in parallel for every task and use whichever one finishes first as the source of truth.
- D. Restart the task from the beginning whenever a process is interrupted.

✓ **Correct Answer: B. Apply a checkpointing pattern that persists the agent's intermediate state and reloads that state when resuming an interrupted task.**

Why: Checkpointing persists intermediate state so a long-running task can resume from the last known-good point instead of starting over. Restarting from scratch (D), running duplicate parallel copies (C), or just extending timeouts (A) don't actually give the agent the ability to resume.

Q014

SCENARIO · SINGLE CHOICE

Your Claude agent has access to a tool that retrieves customer records. A teammate has noticed that the agent occasionally calls the tool with arguments the schema does not declare, and the tool's downstream service returns an error each time. The teammate proposes loosening the schema so the tool accepts whatever arguments the model produces. How would you respond?

- A. Add a system prompt instruction telling the model to produce schema-conforming arguments, treating the prompt instruction as the primary mechanism for keeping the agent's tool calls valid.
- ✓ **B. Keep the schema strict, validate arguments before dispatching, and return a structured error so the agent can**

retry.

- C. Remove the schema entirely and rely on the downstream service to reject invalid calls, treating the downstream service as the team's primary enforcement layer.
- D. Loosen the schema as the teammate proposed so the downstream service receives every call the agent makes during normal operation.

✓ **Correct Answer: B. Keep the schema strict, validate arguments before dispatching, and return a structured error so the agent can retry.**

Why: Keeping the schema strict and validating arguments before dispatch turns malformed calls into a structured, recoverable error the agent can retry against, rather than letting invalid data reach (and break) the downstream service (C, D) or relying solely on a prompt instruction to keep the model honest (A).

Q015

SCENARIO · SINGLE CHOICE

Your Claude application runs long agentic workflows where the agent makes many tool calls, and the conversation history grows quickly. After about 20 tool calls, you notice the agent's responses become less focused and sometimes ignore earlier task constraints. How would you address this?

- A. Remove tool calling from the workflow entirely so the agent operates as a single text-generation step with no tool outputs accumulating in the context window.
- ✓ **B. Apply context engineering techniques such as tool output pruning or compaction to keep the active task state visible while reducing the volume of older content.**
- C. Increase the model's context window so the agent can hold every tool output at full detail across the entire workflow no matter how many tool calls it accumulates.
- D. Restart the agent every five tool calls to prevent any drift, with the agent losing all task state at each restart point during the workflow.

✓ **Correct Answer: B. Apply context engineering techniques such as tool output pruning or compaction to keep the active task state visible while reducing the volume of older content.**

Why: As tool-call history accumulates, pruning or compacting older tool outputs keeps the active task state visible without losing the working context. Removing tool calling (A) breaks the workflow, unbounded context growth (C) doesn't scale, and resetting every five calls (D) throws away needed state.

Q016

SCENARIO · SINGLE CHOICE

Your Claude agent performs database operations. A recent incident occurred where the agent ran a destructive query that affected production data. The team wants to add deterministic controls to prevent similar incidents. How would you prevent similar incidents?

- A. Run the agent only during business hours when humans are available to monitor its activity, treating the schedule as the primary control mechanism for destructive operations.
- ✓ **B. Add Claude hooks that intercept database operations and apply deterministic checks, such as blocking**

destructive queries or requiring approval, before the queries execute.

C. Switch to a higher-capability Claude model on the grounds that a more capable model is less likely to run destructive queries during normal operation across all requests.

D. Add a system prompt instruction telling the agent to be careful with database operations on every request the application handles during normal operation across all incoming traffic.

✓ **Correct Answer: B. Add Claude hooks that intercept database operations and apply deterministic checks, such as blocking destructive queries or requiring approval, before the queries execute.**

Why: Hooks give a deterministic, code-level gate on database operations — blocking destructive queries or requiring approval — which a prompt instruction (D) can't guarantee and a bigger model (C) doesn't guarantee either. Restricting operating hours (A) doesn't prevent the underlying risk.

Q017

SCENARIO · SINGLE CHOICE

The team is debating whether to build a new capability as a custom tool or to use an existing built-in tool that nearly covers the use case but lacks one specific feature. How would you decide?

✓ **A. Evaluate whether the missing feature can be addressed by extending the built-in tool or warrants a custom tool, then choose accordingly.**

B. Identify whether the team has previously built similar custom tools and use those precedents to determine which approach is more consistent with the existing codebase.

C. Examine the custom tool option by scoping the implementation effort and determining whether the additional flexibility justifies the development and maintenance cost.

D. Review the built-in tool's documentation and assess whether its existing capabilities are sufficient to cover the use case without any modification or extension.

✓ **Correct Answer: A. Evaluate whether the missing feature can be addressed by extending the built-in tool or warrants a custom tool, then choose accordingly.**

Why: The decision should hinge on whether the missing feature can be added to the existing built-in tool or genuinely requires a new custom tool — not team familiarity with prior tools (B), a blanket cost/benefit review divorced from the missing feature (C), or assuming the built-in tool is sufficient without checking (D).

Q018

SCENARIO · SINGLE CHOICE

You are reviewing an architectural diagram for a Claude-powered travel-booking system. The diagram shows a top-level component that interprets user requests and three subordinate components that handle flights, hotels, and ground transportation. The top-level component is responsible for routing each request, sequencing the subordinate components, and reconciling their outputs into a final itinerary. The diagram also shows that each subordinate component has its own tool list and own short conversation history that is not shared with the others. Which architectural pattern does this diagram most closely describe?

✓ **A. A manager and supervisor pattern with isolated context per subagent, where the top-level agent coordinates specialized subagents that each maintain their own conversation history and tool list.**

B. A manager and supervisor pattern with shared context, where the top-level agent and the subagents all share a single conversation history that grows as the request flows through the system.

C. A pipeline pattern, where each component processes the user request in sequence and passes the full conversation history along with the request to the next component in the pipeline.

D. A retrieval-augmented pattern, where the top-level component retrieves relevant context from the subordinate components' indexed data stores before generating each response.

✓ **Correct Answer: A. A manager and supervisor pattern with isolated context per subagent, where the top-level agent coordinates specialized subagents that each maintain their own conversation history and tool list.**

Why: A top-level agent that routes, sequences, and reconciles output from separate subagents — each with its own isolated tool list and conversation history — is the manager/supervisor pattern with isolated subagent context. Shared history (B), a linear pipeline (C), and RAG (D) all describe different architectures.

Q019

SCENARIO · SINGLE CHOICE

You are configuring Claude Code for a new project. The team needs to set permissions, default model selections, and environment-specific behavior at the project level so the configuration is consistent across all developers working on the repository. The Claude Code mechanism you would use is...

A. A shared spreadsheet that lists configuration values for team members to reference and update by hand as the project evolves.

B. Environment variables that each developer sets on their own machine when working with Claude Code on the project.

C. A system prompt embedded in every Claude Code conversation by each developer at the start of every session in the project.

✓ **D. The settings.json file, scoped at the project level so the configuration applies consistently across developers and persists with the repository.**

✓ **Correct Answer: D. The settings.json file, scoped at the project level so the configuration applies consistently across developers and persists with the repository.**

Why: settings.json scoped at the project level is the mechanism built for exactly this: consistent, repository-level configuration (permissions, model defaults, environment behavior) that travels with the repo for every developer. Per-developer environment variables (B) or manual documentation (A, C) don't guarantee consistency.

SECTION 2

Questions 20–38

Claude Certified Developer — Foundations · Practice Set

Q020

SCENARIO · SINGLE CHOICE

Your agent makes 10 to 15 tool calls per task, and you have noticed it sometimes loses track of earlier results by the time it reaches later steps. The agent's context window is large enough to hold all the messages, but the relevant information appears to get buried as the conversation grows. How would you address this?

- A. Switch to a different agentic framework that advertises automatic context-window management as a built-in feature.
- B. Reduce the number of tool calls per task by combining several existing tools into larger, multi-purpose tools.
- C. Increase the context window further so all tool outputs from every prior step remain in full detail throughout the task.

✓ **D. Apply a context-management pattern that summarizes or prunes older tool outputs while preserving the active task state.**

✓ **Correct Answer: D. Apply a context-management pattern that summarizes or prunes older tool outputs while preserving the active task state.**

Why: Summarizing or pruning older tool outputs while keeping active task state visible directly addresses information getting buried in a growing context — simply enlarging the context window (C) keeps everything at full detail and doesn't solve the buried-information problem, and switching frameworks (A) or merging tools (B) doesn't address the root cause.

Q021

SCENARIO · SINGLE CHOICE

Your Claude application makes high-volume API calls during business hours and very few calls overnight. The team is concerned about staying within rate limits during peak hours and wants to understand how the Claude API enforces those limits. How would you proceed?

- A. Review the API documentation for streaming endpoints and evaluate whether migrating peak-hour calls to streaming reduces exposure to rate limit enforcement.
- B. Assess the average payload size of current API calls and consolidate requests where possible to reduce the total number of calls made during peak hours.

✓ **C. Identify the rate limits, design the application to stay within them during peak hours, and use exponential**

backoff when limits are reached.

D. Examine the peak-hour request patterns in your application logs and smooth traffic by distributing requests more evenly across the business-hours window.

✓ **Correct Answer: C. Identify the rate limits, design the application to stay within them during peak hours, and use exponential backoff when limits are reached.**

Why: The right approach is to know the actual rate limits, design usage to stay under them during peak periods, and add exponential backoff for the cases where limits are still hit. Streaming (A) and payload consolidation (B, D) may help incidentally but don't address rate-limit compliance directly.

Q022

SCENARIO · SINGLE CHOICE

Your Claude application processes 50-page legal contracts and produces summaries with citation references back to the source. The team is debating whether to send each contract whole or split it into smaller pieces. The contracts fit within Claude's context window. Initial testing shows that whole-document processing produces summaries with stronger cross-section reasoning but occasionally drifts on citation accuracy in later sections. Chunked processing produces stronger citation accuracy per chunk but loses cross-section reasoning. The team has not decided which property matters more. How would you guide the team's decision?

A. Review the citation accuracy results from chunked processing and determine whether the loss of cross-section reasoning produces summaries that still meet the application's quality bar.

✓ **B. Identify which property matters more for the application's actual use case and let that decision drive the chunking approach, then validate the choice against representative contracts.**

C. Examine the use cases where cross-section reasoning failures occur and assess whether whole-document processing meets the application's accuracy requirements across a representative sample of contracts.

D. Assess the cost and latency implications of both approaches against the application's performance requirements before recommending which processing strategy to adopt.

✓ **Correct Answer: B. Identify which property matters more for the application's actual use case and let that decision drive the chunking approach, then validate the choice against representative contracts.**

Why: Whole-document vs. chunked processing is a real tradeoff between cross-section reasoning and citation accuracy, so the decision should be driven by which property the specific use case needs most, validated against real contracts — not decided by looking at only one side's results in isolation (A, C) or by cost/latency alone (D).

Q023

SCENARIO · SINGLE CHOICE

Your Claude application's token costs have grown faster than expected. The team has not been tracking token usage by feature, so the team cannot identify which features are driving cost. The team is debating how to respond. How would you respond?

A. Tell the team that cost growth is unavoidable as the application scales and that no investigation will change the

trajectory of the application's cost over the next several quarters.

- B. Switch every feature to the smallest model to cut cost broadly across the application during normal operation.
- C. Reduce token usage uniformly across all features by half, applying the cut evenly across the application during normal operation.

✓ D. Add token usage tracking by feature to the application's logging so the team can identify which features drive cost before recommending changes.

✓ **Correct Answer: D. Add token usage tracking by feature to the application's logging so the team can identify which features drive cost before recommending changes.**

Why: You can't optimize what you can't measure — adding per-feature token usage tracking gives the team the data needed to find out what is actually driving cost before making changes. Uniform cuts (C), switching every feature to the smallest model (B), or assuming nothing can be done (A) all skip the diagnostic step.

Q024

SCENARIO · SINGLE CHOICE

Your Claude application's outputs are inconsistent in format. The team's instructions are scattered across the system prompt and user messages, with some instructions stated only once and others repeated. How would you fix the inconsistency?

- A. Move all instructions into the user message so the application's behavior is controlled at the input point.

✓ B. Consolidate behavioral and format instructions in the system prompt and keep user messages focused on the user's input.

- C. Place each instruction wherever the developer who first wrote it found most natural, retaining the original intent.
- D. Repeat all instructions in both the system prompt and the user message so the model sees each instruction more than once.

✓ **Correct Answer: B. Consolidate behavioral and format instructions in the system prompt and keep user messages focused on the user's input.**

Why: Consolidating format and behavioral instructions into the system prompt, with user messages left focused on user input, gives the model one consistent, authoritative source of formatting rules. Scattering instructions across user messages (A), leaving placement to individual developer preference (C), or duplicating instructions everywhere (D) is what caused the inconsistency in the first place.

Q025

SCENARIO · SINGLE CHOICE

Your team is debating how to manage the prompts used in your Claude application. Some prompts are checked into the code repository, some live in a separate configuration file, and some are constructed inline at runtime. The result is inconsistent, and a recent prompt change went out without code review. What steps would you take?

- A. Move all prompts out of version control to a separate spreadsheet that team members can edit freely as the

application evolves over time.

B. Move all prompts to inline runtime construction so the team can update them quickly through a streamlined process outside the standard code review workflow.

C. Establish a single source of truth for prompts but keep change review optional, allowing developers to update prompts directly when changes are urgent.

✓ **D. Establish a single source of truth for prompts, version-control them alongside code, and require code review for prompt changes.**

✓ **Correct Answer: D. Establish a single source of truth for prompts, version-control them alongside code, and require code review for prompt changes.**

Why: Prompts are part of the application's logic and deserve the same treatment as code: a single source of truth, version control, and mandatory review. Moving them further from that discipline — a spreadsheet (A), inline runtime construction (B), or optional review (C) — is what let an unreviewed change ship.

Q026

SCENARIO · SINGLE CHOICE

Your Claude application validates structured output but has been treating validation failures as terminal errors. Each validation failure causes the entire user request to fail. The team wants to handle validation failures more gracefully. How would you handle the validation failures?

A. Pass validation failures directly to downstream systems and let each downstream system decide how to handle the malformed output.

B. Disable output validation until the underlying cause of validation failures has been identified and addressed in a future release.

✓ **C. Treat validation failures as a recognized error path that triggers retry, repair, or fallback logic before failing the user request.**

D. Tell users that validation failures are unavoidable and instruct them to perform manual accuracy checks before relying on outputs.

✓ **Correct Answer: C. Treat validation failures as a recognized error path that triggers retry, repair, or fallback logic before failing the user request.**

Why: Treating validation failures as a recognized, expected error path — with retry, repair, or fallback logic — is far more resilient than failing the whole request (the original problem), silently disabling validation (B), or pushing the raw failure downstream (A, D).

Q027

SCENARIO · SINGLE CHOICE

You are extending a Claude agent with a capability that needs to be reusable across multiple teams in the organization, with each team able to invoke and use it independently. How would you build the capability?

A. As a custom tool embedded in this team's agent only, with other teams able to copy the implementation into

their own agents when they need the capability.

B. As a shared library that each team imports into its own Claude application code, with each team responsible for keeping the library up to date in its integration.

✓ **C. As a Skill or MCP server because both are purpose-built for cross-team reuse independently by each consuming team.**

D. As a wrapper around an existing built-in tool that adds the missing functionality, on the grounds that built-in tools cover the reuse pattern when extended carefully.

✓ **Correct Answer: C. As a Skill or MCP server because both are purpose-built for cross-team reuse independently by each consuming team.**

Why: Skills and MCP servers are the mechanisms purpose-built for capabilities that need to be discovered and invoked independently by multiple teams. Embedding the capability in one team's agent (A), a shared library each team has to manually keep in sync (B), or a narrow built-in tool wrapper (D) don't give the same clean, independent reusability.

Q028

SCENARIO · SINGLE CHOICE

A Claude application is occasionally refusing to answer questions that should be in scope, including questions the application has answered correctly in the past. You want to investigate. What is the first step of your investigation?

A. Test whether removing or adjusting those instructions resolves the behavior.

✓ **B. Examine traces of the refused requests to identify what triggers the refusal, whether input patterns, system prompt content, or other context.**

C. Assess whether the refusals cluster around specific input patterns, question types, or time periods by analyzing the distribution of refused requests in the logs.

D. Identify which questions have been refused and compare them against previously answered questions to determine what changed between the successful and refused interactions.

✓ **Correct Answer: B. Examine traces of the refused requests to identify what triggers the refusal, whether input patterns, system prompt content, or other context.**

Why: The first step in any investigation like this is to look at traces of the actual refused requests to find the trigger — input pattern, prompt content, or other context — before guessing at a fix. Jumping straight to removing instructions (A) or comparing requests without first understanding the trigger (C, D) skips the diagnostic step.

Q029

SCENARIO · SINGLE CHOICE

The team is debating whether to integrate with the Claude API directly or through a third-party abstraction layer that supports multiple LLM providers. The team has identified that all current and projected use cases run on Claude, no internal customer has requested LLM portability, and the team's product roadmap does not mention multi-provider support over the next two years. The third-party abstraction would add roughly 15 percent overhead in code complexity and introduce one additional

dependency. Which integration approach would you recommend?

- A. The third-party abstraction layer, on the grounds that multi-provider support is valuable for any application as a matter of long-term flexibility across vendors.
- B. Both integration paths in parallel, where the application uses each path on different runs to compare which performs better in production over time.
- C. A custom multi-provider abstraction layer the team builds in-house so that the team controls every part of the abstraction the application uses for its API calls.
- ✓ D. Direct integration with Claude through its SDK, because no multi-provider need exists and abstraction would add complexity that does not pay off.

✓ **Correct Answer: D. Direct integration with Claude through its SDK, because no multi-provider need exists and abstraction would add complexity that does not pay off.**

Why: With no multi-provider requirement on the roadmap, no customer asking for portability, and real added complexity, direct integration with the Claude SDK is the pragmatic choice — the abstraction layer (A) or a custom in-house one (C) would be paying a real cost for flexibility nobody needs, and running both in parallel (B) adds cost without benefit.

Q030

SCENARIO · SINGLE CHOICE

Your team is choosing how to add a capability to a Claude application. You want to apply the appropriate option, whether built-in tool, custom tool, Skill, or MCP server, based on the use case. You would choose the option that...

- A. Is the most familiar to the team based on prior experience with similar capabilities.
- B. Is the newest available option among built-in tools, custom tools, Skills, and MCP servers.
- C. Is the easiest to build given the team's current development tools and workflows.
- ✓ D. Matches the use case's scope, reuse needs, and integration context.

✓ **Correct Answer: D. Matches the use case's scope, reuse needs, and integration context.**

Why: The right extension mechanism (built-in tool, custom tool, Skill, or MCP server) is the one that matches the specific use case's scope, reuse needs, and integration context — not what's most familiar (A), newest (B), or easiest to build with current tooling (C), which are convenience factors rather than fit factors.

Q031

SCENARIO · SINGLE CHOICE

You are designing a Claude application that processes user-submitted text. Some of that text could include sensitive information such as account numbers or passwords that the application should not send to Claude. How would you design the application?

- ✓ A. Define the application boundary explicitly, identify what content can leave the boundary for Claude, and add filtering or redaction at the boundary.

- B. Add a prompt instruction in the system prompt specifying the categories of sensitive information Claude should disregard when processing user-submitted text.
- C. Log all user-submitted text before it is sent to Claude and review the logs periodically to identify whether sensitive information is reaching the model.
- D. Apply filtering at the boundary for the most commonly observed sensitive data patterns and expand coverage to additional patterns based on findings from production monitoring.

✓ **Correct Answer: A. Define the application boundary explicitly, identify what content can leave the boundary for Claude, and add filtering or redaction at the boundary.**

Why: Defining the application boundary explicitly and filtering or redacting sensitive content before it crosses that boundary keeps sensitive data like account numbers or passwords from ever reaching Claude. Relying on the model to disregard sensitive content per a prompt instruction (B) or only reviewing logs after the fact (C) doesn't prevent exposure.

Q032

SCENARIO · SINGLE CHOICE

The Anthropic API deprecated a request parameter that your Claude application uses in approximately 40 places across the codebase. The deprecation notice gives a six-month window before the parameter is removed and recommends a replacement parameter with slightly different semantics. You would respond to the deprecation by...

- A. Migrating all 40 call sites in a single change near the removal date to ensure the deprecated parameter continues to work as long as possible.
- B. Keeping the deprecated parameter in place while writing a wrapper function around it to insulate the rest of the codebase from the eventual change.

✓ **C. Adding regression tests to cover the parameter's behavior, then migrating call sites in batches that you validate against regression tests.**

- D. Swapping all 40 call sites in a single change right away to prevent drawn-out migration work that will delay ongoing functioning.

✓ **Correct Answer: C. Adding regression tests to cover the parameter's behavior, then migrating call sites in batches that you validate against regression tests.**

Why: For a widely-used parameter with a real deprecation window, the safe path is to add regression tests first and then migrate call sites in validated batches — a single last-minute cutover (A, D) risks a broad, hard-to-debug break, and a permanent wrapper (B) just delays the inevitable migration.

Q033

SCENARIO · SINGLE CHOICE

You are explaining to a stakeholder why running the same Claude prompt twice can produce slightly different results. The stakeholder is concerned this means the application is broken. How would you address the stakeholder's concern?

- A. Tell the stakeholder the variation is a bug that the team will fix in the next release of the application, then create

a work ticket to fix the bug.

B. Tell the stakeholder the variation comes from network latency and that switching to a faster network connection will produce more consistent results across runs.

✓ **C.** Explain that LLMs are non-deterministic by default due to sampling, and describe how the application handles this through validation, retries, or temperature adjustment.

D. Tell the stakeholder the variation is caused by Claude being updated continuously by Anthropic, and that switching to a fixed model snapshot will eliminate the variation entirely.

✓ **Correct Answer: C.** Explain that LLMs are non-deterministic by default due to sampling, and describe how the application handles this through validation, retries, or temperature adjustment.

Why: LLM sampling is non-deterministic by default, so explaining that mechanism — and how the application handles it through validation, retries, or temperature settings — is the accurate response. Calling it a bug (A), blaming network latency (B), or blaming continuous model updates (D) are all incorrect explanations.

Q034

SCENARIO · SINGLE CHOICE

Your team is preparing a new Claude application for production, and the product team has asked for a cost projection. The team needs to estimate the cost based on expected request volume, average input length, and average output length. How would you build the projection?

A. Build a cost model that uses the average per-request cost from a similar Claude application the team built last year, scaling that figure by expected request volume.

✓ **B.** Build a cost model that combines expected request volume, average input tokens, average output tokens, the chosen model's pricing, and any caching benefits.

C. Build a cost model that combines expected request volume and average input tokens, treating output tokens as a small enough share of cost to leave out of the projection.

D. Build a cost model based on expected request volume and the chosen model's pricing, treating average input and output token counts as variables to be estimated post-launch.

✓ **Correct Answer: B.** Build a cost model that combines expected request volume, average input tokens, average output tokens, the chosen model's pricing, and any caching benefits.

Why: A real cost projection needs to combine expected volume, average input and output tokens, the specific model's pricing, and any caching benefits. Estimating from a different application's per-request average (A), ignoring output tokens (C), or deferring token estimates to after launch (D) all produce an unreliable projection.

Q035

SCENARIO · SINGLE CHOICE

Your Claude application receives untrusted input from external sources. The team is establishing how the application should treat this untrusted input. Untrusted input would be...

✓ **A.** Validated and sanitized before being incorporated into prompts, and treated as data the model should not

interpret as instructions.

- B.** Treated identically to trusted input from internal sources, on the grounds that all input the application receives can be handled the same way.
- C.** Routed through a separate Claude application with more lax security controls so the original application does not handle the untrusted input directly.
- D.** Blocked entirely so the application accepts only trusted input, with no path for legitimate external input the application might process.

✓ **Correct Answer: A. Validated and sanitized before being incorporated into prompts, and treated as data the model should not interpret as instructions.**

Why: Untrusted input should be validated, sanitized, and treated as data rather than as instructions the model should follow — this is the core defense against prompt injection. Treating it identically to trusted input (B), routing it to a less-secure app (C), or blocking it outright (D) either ignores the risk or throws away legitimate use cases.

Q036

SCENARIO · SINGLE CHOICE

You are writing a system prompt for a Claude application that needs to produce output in a specific JSON shape. The downstream system will reject any output that does not match the schema. Your prompt would need to...

- A.** Instruct Claude to use whichever output format it considers most appropriate for each request the application handles.
- B.** Instruct Claude to return JSON sometimes and free text other times so the application's output captures both formats.
- C.** Omit any reference to the format and rely on a post-processing step in the application to reshape Claude's output.

✓ **D. Include explicit constraints describing the required JSON schema and an instruction to produce only output matching that schema.**

✓ **Correct Answer: D. Include explicit constraints describing the required JSON schema and an instruction to produce only output matching that schema.**

Why: When a downstream system will reject anything that doesn't match a schema, the prompt needs to explicitly spell out that schema and instruct the model to produce only conforming output. Letting the model choose its own format (A), mixing formats (B), or relying entirely on post-processing to fix the shape (C) all risk rejected output.

Q037

SCENARIO · SINGLE CHOICE

Your Claude application's error handling currently logs every API error with the same severity level. The team wants to differentiate between errors that should page an on-call engineer and errors that should be logged for later review. How would you structure the error handling?

- A.** Page on every error, on the grounds that paging guarantees that no error is missed by the team during normal

operation across the application's lifecycle.

B. Disable logging for any error that does not page, treating non-paging errors as not worth recording for later review either.

✓ **C. Categorize errors by severity based on impact and recoverability, then route each category to the appropriate channel for paging or logging.**

D. Log every error with the same severity, on the grounds that differentiating severity adds complexity that does not pay off in most application setups over time.

✓ **Correct Answer: C. Categorize errors by severity based on impact and recoverability, then route each category to the appropriate channel for paging or logging.**

Why: Categorizing errors by impact and recoverability, then routing each category to the right channel, is what lets a team distinguish genuine on-call emergencies from things that just need to be logged for review. Paging on everything (A) causes alert fatigue, and treating all errors identically (B, D) loses that distinction entirely.

Q038

SCENARIO · SINGLE CHOICE

You are building an agent that needs to call several internal APIs and a database in a structured, repeatable way. Your team has decided to use the Claude Agent SDK rather than build a custom loop. You are setting up the agent's tool definitions and execution loop. How would you set up the tools and execution loop?

✓ **A. Use the SDK's tool interface and let the SDK handle the loop, dispatch, and history.**

B. Call the Messages API directly and let the model format its tool calls in plain text.

C. Use the SDK's tool interface and loop, with conversation history stored in a separate team database.

D. Use the SDK's tool interface and write the loop and history layer in the team's own code.

✓ **Correct Answer: A. Use the SDK's tool interface and let the SDK handle the loop, dispatch, and history.**

Why: Once the team has chosen the Agent SDK specifically to avoid building a custom loop, the SDK's tool interface should also handle dispatch and history — that's the point of using it. Formatting tool calls as plain text (B) or building a separate history layer (C, D) undermines the reason for choosing the SDK.

SECTION 3

Questions 39–57

Claude Certified Developer — Foundations · Practice Set

Q039

SCENARIO · SINGLE CHOICE

Your Claude application has been running for several conversation turns, and you notice the model occasionally references information that was discussed many turns ago but is no longer relevant. You suspect context drift is causing the model to weight stale content too heavily. How would you address the drift?

- A. Increase the context window size so all turns of the conversation remain visible to the model in full detail.
- B. Reset the conversation after every turn so the model loses all prior turns when generating a response.

✓ C. Apply compaction to summarize older portions of the conversation so the gist remains while the specifics carry less weight.

- D. Truncate the conversation so the model sees only the most recent turn during each subsequent response.

✓ **Correct Answer: C. Apply compaction to summarize older portions of the conversation so the gist remains while the specifics carry less weight.**

Why: Compaction summarizes older parts of the conversation so the gist is preserved while it takes up less weight relative to the current turn, directly countering context drift. Growing the window indefinitely (A), resetting every turn (B), or truncating to only the latest turn (D) either don't scale or throw away needed continuity.

Q040

SCENARIO · SINGLE CHOICE

Your Claude application uses structured output that is consumed by downstream code. The team wants to handle malformed or unexpected output gracefully so it does not crash downstream systems. The best choice for handling this issue would be to...

- A. Crash on any unexpected output and require manual recovery on affected requests the application handles during normal operation.
- B. Silently drop any output that does not match expectations so downstream systems do not see the malformed output the application receives from the model.
- C. Use the output the model produces and leave schema enforcement to a later phase of production after the application is more complete.

✓ D. Apply defensive parsing to handle unexpected fields, missing values, and type mismatches that the

downstream code might otherwise crash on.

✓ **Correct Answer: D. Apply defensive parsing to handle unexpected fields, missing values, and type mismatches that the downstream code might otherwise crash on.**

Why: Defensive parsing that explicitly handles unexpected fields, missing values, and type mismatches keeps malformed model output from crashing downstream code. Crashing outright (A), silently dropping output (B), or deferring schema enforcement to later (C) all leave the downstream system exposed in the meantime.

Q041

SCENARIO · SINGLE CHOICE

Your team uses Claude Code across multiple repositories. You want the team's rules and general coding standards to apply to all repositories, and other rules to apply only to specific repositories. The team is currently duplicating instructions across every repository's CLAUDE.md file. How would you address this?

A. Move all instructions to a separate documentation site that developers consult during Claude Code sessions across all repositories.

✓ **B. Use a CLAUDE.md hierarchy that scopes general standards broadly and project-specific context within each repository's local CLAUDE.md.**

C. Stop using CLAUDE.md altogether and ask each developer to configure Claude Code manually for each project they work on.

D. Use a single repository's CLAUDE.md as the central source of truth and link to it from every other repository's CLAUDE.md file.

✓ **Correct Answer: B. Use a CLAUDE.md hierarchy that scopes general standards broadly and project-specific context within each repository's local CLAUDE.md.**

Why: A CLAUDE.md hierarchy lets broad team-wide standards live at a shared level while project-specific context stays local to each repository, eliminating the duplication problem. A separate documentation site (A), abandoning CLAUDE.md (C), or funneling every repo through one central file (D) don't give the same clean scoping.

Q042

SCENARIO · SINGLE CHOICE

Your team uses several plugins across multiple Claude applications, and a recent plugin update introduced a regression. The team had not been tracking plugin versions, so the team cannot easily identify which version was previously working. How would you address this?

A. Stop using all plugins until the team can rebuild equivalent functionality directly into the application code, treating plugin avoidance as a way to remove version-related risk.

✓ **B. Add explicit plugin version tracking to the project's configuration so the team can identify, pin, and upgrade plugin versions deliberately.**

C. Treat plugins as untrackable third-party code and rely on plugin authors to communicate breaking changes when they happen, with no internal version tracking.

D. Upgrade every plugin to the latest version on a regular cadence to keep version drift small, on the grounds that drift contributes to regression risk.

✓ **Correct Answer: B. Add explicit plugin version tracking to the project's configuration so the team can identify, pin, and upgrade plugin versions deliberately.**

Why: Explicit plugin version tracking in project configuration is what lets a team identify, pin, and deliberately upgrade versions — which is exactly the capability missing when a regression can't be traced to a specific update. Avoiding plugins altogether (A), treating them as untrackable (C), or auto-upgrading everything regularly (D) don't solve the underlying tracking gap.

Q043

SCENARIO · SINGLE CHOICE

Your Claude application uses tool calling to fetch patient data and generate summary reports. The flow occasionally fails because the model returns a `tool_use` block that references arguments not present in the schema, and your application code does not handle this case gracefully. How would you address this?

✓ **A. Validate the `tool_use` block's arguments against the tool schema before dispatching the tool and handle invalid arguments as a recognized error path.**

B. Log invalid `tool_use` blocks when they occur and allow the tool dispatch to proceed, relying on the tool's own error handling to surface failures back to the application.

C. Retry the same request repeatedly until the model returns a valid `tool_use` block that matches the schema as expected.

D. Stop using tool calling entirely and replace tools with prompted text generation that asks the model to describe what it would do.

✓ **Correct Answer: A. Validate the `tool_use` block's arguments against the tool schema before dispatching the tool and handle invalid arguments as a recognized error path.**

Why: Validating the `tool_use` block's arguments against the schema before dispatch turns an invalid call into a recognized, handleable error instead of letting it silently break the flow. Logging and dispatching anyway (B), retrying blindly (C), or abandoning tool calling (D) don't actually fix the mismatch.

Q044

SCENARIO · SINGLE CHOICE

You are designing a Claude application that maintains user sessions across multi-turn conversations. The product team has asked how the application will handle session lifecycle: when sessions should expire, how state is reset, and how the application avoids carrying stale context into new conversations. How would you design session lifecycle?

✓ **A. Define explicit session expiration rules, state reset triggers, and rules for starting fresh sessions so stale context does not leak into new conversations.**

B. Define a single short session timeout that applies across all conversations and treat the timeout as the application's complete session lifecycle mechanism.

- C. Define explicit session expiration rules but rely on users to start new conversations when they want fresh context, with no automatic reset triggers in the application.
- D. Define state reset triggers tied to specific application events but apply them across all sessions globally, with no per-session expiration rules.

✓ **Correct Answer: A. Define explicit session expiration rules, state reset triggers, and rules for starting fresh sessions so stale context does not leak into new conversations.**

Why: Explicit session expiration rules, state reset triggers, and rules for starting fresh sessions together prevent stale context from leaking into new conversations. A single blanket timeout (B), relying on users to reset state themselves (C), or global reset triggers with no per-session rules (D) each cover only part of the lifecycle.

Q045

SCENARIO • SINGLE CHOICE

You are building an MCP server that exposes several internal data sources as MCP resources. The server needs to be deployed so multiple Claude applications can integrate with it. How would you approach the build and deployment?

✓ **A. Author the server with clearly defined resources, tools, and prompts, choose a communication pattern, and deploy to an accessible hosting environment.**

- B. Build the MCP server with resource and tool definitions scoped to the first Claude application that needs it, and extend the definitions to additional applications as each integration is requested.
- C. Deploy the MCP server only on individual developer machines, with the Claude applications unable to reach the server outside each developer's machine.
- D. Bypass the MCP server and embed each data source directly in every Claude application that needs the data, with each application maintaining its own integration.

✓ **Correct Answer: A. Author the server with clearly defined resources, tools, and prompts, choose a communication pattern, and deploy to an accessible hosting environment.**

Why: A reusable MCP server needs clearly defined resources, tools, and prompts, a deliberate communication pattern, and deployment somewhere multiple applications can actually reach it. Scoping it to just the first consumer (B), keeping it local to individual machines (C), or skipping the server and duplicating integrations per application (D) undermines the point of building a shared server.

Q046

SCENARIO • SINGLE CHOICE

You are deciding between Claude models for a task. The team has identified three relevant tradeoff dimensions: quality, latency, and cost. The right model is the one that...

- A. Satisfies the task's latency requirement first, then is evaluated against quality and cost thresholds to confirm the selection is acceptable across all three dimensions.
- B. Meets the task's cost target within a defined latency budget, with quality validated against a representative sample of inputs after the model is selected.

✓ **C. Fits the task's quality, latency, and cost requirements together, recognizing that improving one dimension typically affects the others.**

D. Meets the task's quality requirements at an acceptable latency, with cost reviewed separately once the quality and latency bar has been established.

✓ **Correct Answer: C. Fits the task's quality, latency, and cost requirements together, recognizing that improving one dimension typically affects the others.**

Why: Quality, latency, and cost interact — improving one typically affects the others — so the right model is chosen by fitting all three requirements together rather than optimizing one dimension first and checking the others after the fact (A, B, D).

Q047

SCENARIO · SINGLE CHOICE

A teammate has asked how the Claude SDK handles transient API errors, such as a temporary network issue or a brief rate-limit response. They want to know whether the application code needs to handle every transient error or whether the SDK provides any default behavior. How would you describe the SDK's default behavior?

A. The SDK provides default retry behavior for transient errors up to a fixed number of attempts, and this behavior is not configurable.

B. The SDK provides default retry behavior for network errors but surfaces rate-limit responses directly to the application code, which must implement its own retry logic for those cases.

C. The SDK logs transient errors to a default error stream and continues execution without retrying, leaving the application code responsible for detecting and responding to failed calls.

✓ **D. The SDK provides default retry behavior for many transient errors, and the application code can configure or extend that behavior as needed.**

✓ **Correct Answer: D. The SDK provides default retry behavior for many transient errors, and the application code can configure or extend that behavior as needed.**

Why: The Claude SDK provides default retry behavior for many transient errors out of the box, and that behavior is configurable and extendable by application code — it isn't a fixed, non-configurable count (A), it doesn't skip rate-limit handling (B), and it doesn't just log and give up (C).

Q048

SCENARIO · SINGLE CHOICE

A Claude application that worked well in testing is now occasionally returning outputs that mention information not present in the input. The development team initially assumed the model was hallucinating, so they asked you to troubleshoot. What would you do first?

✓ **A. Examine production traces to identify whether the issue is hallucination by the model, context loss, prompt injection, or another failure mode before recommending a fix.**

B. Replace the current model with a larger one to reduce the chance of hallucination, on the grounds that larger

models tend to hallucinate less in typical applications.

C. Apply a retrieval-augmented generation pattern to ground the responses in source content before any further investigation of the production traces.

D. Add a system prompt instruction telling the model not to invent information, on the grounds that prompt-level instructions are the fastest fix for hallucination concerns.

✓ **Correct Answer: A. Examine production traces to identify whether the issue is hallucination by the model, context loss, prompt injection, or another failure mode before recommending a fix.**

Why: The right first step is to look at production traces to determine the actual failure mode — hallucination, context loss, prompt injection, or something else — before picking a fix. Assuming it's hallucination and reaching for a bigger model (B), adding RAG (C), or a prompt instruction (D) all skip that diagnostic step.

Q049

SCENARIO • SINGLE CHOICE

A Claude application is producing outputs that drift away from the expected JSON format after several conversation turns. The first few turns produce correctly formatted output, but later turns gradually lose structure. How would you address the drift?

✓ **A. Identify the failure mode as format drift, examine how the conversation context evolves over turns, and address the drift through context management or output validation.**

B. Truncate every response to the first few characters, validate that the truncated output matches the expected JSON structure, and log any mismatches for review.

C. Restart the application after every turn and monitor whether the format remains consistent across subsequent interactions.

D. Switch to a smaller Claude model and re-test the application to determine whether the drift persists across conversation turns.

✓ **Correct Answer: A. Identify the failure mode as format drift, examine how the conversation context evolves over turns, and address the drift through context management or output validation.**

Why: Naming the failure mode as format drift and examining how context evolves over the conversation lets the team address it with the right tool — context management or output validation. Truncating responses (B), restarting after every turn (C), or just trying a smaller model (D) don't target the actual cause.

Q050

SCENARIO • SINGLE CHOICE

You are setting up a Claude application that will run a mix of multi-turn conversations and one-off requests. You want to use caching techniques to reduce token costs where they apply. A teammate suggests caching the model's output as well, so the application does not have to make duplicate Claude calls when similar queries arrive. You would apply prompt caching to...

A. Nothing, because prompt caching does not affect cost in any application that mixes multi-turn conversations and one-off requests in a single deployment.

B. The model's output, treating the response from each request as cacheable content the application can return on similar future queries.

C. Only the user's input portion of each request because user input is the part of the prompt that varies the most across the application's normal operation.

✓ **D. The static portions of prompts that are repeated across requests, such as system prompts, instructions, or shared context.**

✓ **Correct Answer: D. The static portions of prompts that are repeated across requests, such as system prompts, instructions, or shared context.**

Why: Prompt caching is meant for the static, repeated parts of a prompt — system instructions, shared context — not for caching the model's generated output, which can legitimately vary between similar-looking requests (B). Caching only user input (C) misses the actual reusable content, and dismissing caching altogether (A) ignores its real cost benefit.

Q051

SCENARIO • SINGLE CHOICE

A teammate has asked how to extend Claude Code with a custom Skill that the team can invoke during sessions. The Skill consists of a set of instructions and a few support scripts the team wants Claude to be able to call when the Skill is loaded. Where is the right place to define the Skill?

A. Define the Skill as a long inline instruction at the top of every CLAUDE.md file in the team's repositories so Claude has access to it on every session.

B. Define the Skill inside the application's source code as a regular library module and call it from the application code instead of from Claude Code.

✓ **C. Define the Skill in a Skills directory recognized by Claude Code, where Claude can discover and load it during sessions for the team's repositories.**

D. Define the Skill in a personal scratch directory on each developer's machine and load it manually before each Claude Code session that needs it.

✓ **Correct Answer: C. Define the Skill in a Skills directory recognized by Claude Code, where Claude can discover and load it during sessions for the team's repositories.**

Why: Claude Code looks for Skills in a recognized Skills directory, where it can discover and load the instructions and scripts during a session. Cramming the Skill into every CLAUDE.md (A), building it as an application-only library (B), or keeping it in a personal scratch folder loaded manually (D) don't give the team-wide, discoverable reuse a Skill is meant to provide.

Q052

SCENARIO • SINGLE CHOICE

You are setting up Claude Code for a new project repository. Your team has shared coding standards, preferred libraries, and project-specific context that every developer working on the repository should have available when they use Claude Code. How would you set this up?

A. Document the standards in a separate wiki page maintained outside the repository so the documentation stays decoupled from the source code.

B. Configure Claude Code through environment variables that each developer sets on their own machine when they begin working in the repository.

✓ **C. Initialize Claude Code in the repository and document the standards and project-specific context in a CLAUDE.md file at the repository root.**

D. Add the coding standards and project context to the repository's existing README file and direct developers to reference it when starting Claude Code sessions.

✓ **Correct Answer: C. Initialize Claude Code in the repository and document the standards and project-specific context in a CLAUDE.md file at the repository root.**

Why: Initializing Claude Code in the repo and documenting standards and project context in a root-level CLAUDE.md is the built-for-purpose mechanism, automatically available to every developer in every session. A separate wiki (A), per-developer environment variables (B), or piggybacking on the README (D) don't get picked up the same consistent way.

Q053

SCENARIO • SINGLE CHOICE

Your Claude application is hitting context window limits when processing long customer service transcripts. A junior developer suggests increasing the temperature parameter to fix the issue. How would you respond?

✓ **A. Explain that temperature controls sampling randomness and is unrelated to context capacity, then address the context issue through summarization or chunking.**

B. Adjust the temperature parameter together with the max_tokens parameter, treating the combined adjustment as the team's mechanism for managing context window pressure during long-transcript processing.

C. Remove the system prompt entirely to make room for longer transcripts in each request, freeing up context window space the system prompt would otherwise consume.

D. Increase the temperature parameter as the junior developer suggested and observe whether the context window issue resolves over the next several runs of the application in production.

✓ **Correct Answer: A. Explain that temperature controls sampling randomness and is unrelated to context capacity, then address the context issue through summarization or chunking.**

Why: Temperature controls sampling randomness, not context capacity, so it has nothing to do with hitting context window limits — the actual fix is summarization or chunking of the long transcripts. Combining temperature with max_tokens (B), removing the system prompt (C), or trying the junior developer's suggestion anyway (D) don't address the real constraint.

Q054

SCENARIO • SINGLE CHOICE

You are designing an agent that handles a complex claim-processing workflow. Each claim moves through fact extraction, eligibility evaluation, and a decision step. The three subtasks have distinct success criteria, and some claims require iteration between fact extraction and eligibility evaluation before a decision can be reached. Which agent pattern would you apply?

✓ **A. A graph-based pattern that lets the agent move between subtasks based on the state of each claim, with each**

subtask evaluated against its own criteria.

- B.** A streaming pattern that emits partial decisions as the agent processes each claim, refining the output until a final decision emerges from the stream.
- C.** A single tool-use loop pattern that gives one agent access to all the tools needed for fact extraction, eligibility evaluation, and decision-making.
- D.** A linear chain pattern that processes every claim through fact extraction, then eligibility evaluation, then decision, with no return paths between subtasks.

✓ **Correct Answer: A. A graph-based pattern that lets the agent move between subtasks based on the state of each claim, with each subtask evaluated against its own criteria.**

Why: A graph-based pattern lets the agent move between fact extraction, eligibility evaluation, and the decision step based on the claim's state — including looping back — which fits a workflow that needs iteration between stages. A strictly linear chain with no return paths (D), a single flat tool-use loop (C), or a streaming pattern (B) don't support that back-and-forth.

Q055

SCENARIO · SINGLE CHOICE

Your application uses the Messages API to handle multi-turn conversations. Each new turn resends the entire conversation history, and your token costs are growing as conversations get longer. You suspect there is a more efficient approach. How would you address this?

✓ **A. Use prompt caching to reuse the static portions of the conversation context across turns instead of paying for them at the normal input-token rate on every request the application sends.**

- B.** Switch to the Batch API for every turn so the per-call cost is reduced, treating the batch as the team's primary cost-control mechanism for multi-turn work.
- C.** Truncate every conversation to the last two messages so that token usage stays low and costs remain predictable across the application's normal operation.
- D.** Summarize each conversation after every turn and replace the full history with the summary on the next request, reducing token count at the cost of fidelity.

✓ **Correct Answer: A. Use prompt caching to reuse the static portions of the conversation context across turns instead of paying for them at the normal input-token rate on every request the application sends.**

Why: Prompt caching reuses the static portions of a growing conversation across turns instead of paying full input-token price for them every time, which directly targets rising costs from resending full history. Batch API (B), truncating history (C), or summarizing after every turn (D) either don't fit multi-turn latency needs or sacrifice fidelity unnecessarily.

Q056

SCENARIO · SINGLE CHOICE

A team has deployed a multi-agent system in which a primary agent decomposes user requests and delegates subtasks to three specialized subagents: one for data retrieval, one for analysis, and one for report generation. In production, the team observes that subagents are making redundant tool calls, occasionally exceeding token budgets, and sometimes producing outputs that contradict each other —

all of which the primary agent passes along without catching. What is the most appropriate way to address these failures?

- A. Add retry logic to each subagent so that when a tool call fails, the subagent retries automatically before escalating — and configure each subagent to log its tool calls and outputs to a shared trace so the team can audit redundancy and contradictions after the fact.
- B. Give each subagent read access to the other subagents' outputs so they can identify and resolve contradictions without routing back through the primary agent.
- ✓ C. Strengthen the primary agent's management layer to enforce per-subagent tool budgets, validate outputs against a defined schema before passing them forward, and establish explicit handoff contracts between stages.
- D. Collapse the three subagents into a single large-context model call that handles retrieval, analysis, and generation in one pass.

✓ **Correct Answer: C. Strengthen the primary agent's management layer to enforce per-subagent tool budgets, validate outputs against a defined schema before passing them forward, and establish explicit handoff contracts between stages.**

Why: Strengthening the primary agent's management layer — enforcing per-subagent tool budgets, validating outputs against a schema, and defining explicit handoffs — addresses redundant calls, budget overruns, and contradictions at the coordination layer where they're actually caused. Just adding retries and logging (A), letting subagents read each other's output directly (B), or collapsing everything into one call (D) don't fix the missing oversight.

Q057

SCENARIO · SINGLE CHOICE

The product team has asked you to choose a Claude model for a new feature. The team has provided functional requirements but has not specified performance, cost, or quality targets. The team's product manager says, "Use whatever model gives us the best results." How would you respond?

- ✓ A. Ask the product team to specify quality, latency, and cost targets, then select the model whose tradeoffs best fit those targets.
- B. Run every Claude model on a representative sample and pick whichever scores best on a generic benchmark.
- C. Choose a mid-tier model and ship the feature, because mid-tier models work for most use cases without specified targets.
- D. Choose the largest, highest-capability Claude model, on the grounds that "best results" is most likely to mean highest quality.

✓ **Correct Answer: A. Ask the product team to specify quality, latency, and cost targets, then select the model whose tradeoffs best fit those targets.**

Why: Without specified quality, latency, or cost targets, the right move is to go back and get those targets defined, then select the model whose tradeoffs fit them — not guess based on a generic benchmark (B), default to a mid-tier model (C), or assume 'best' means 'biggest' (D).

SECTION 4

Questions 58–76

Claude Certified Developer — Foundations · Practice Set

Q058

SCENARIO · SINGLE CHOICE

Your team is preparing to roll out a configuration change that updates several prompt versions across a Claude application used by multiple downstream systems. The change has already been tested in staging, but the team has not assessed how the prompt change will affect each downstream system that depends on the application's output. What would you do before rolling out the change?

✓ **A. Assess the configuration impact on each downstream system before rolling out, and coordinate with downstream system owners as needed.**

B. Document the prompt version changes in the application changelog and proceed with the rollout, treating the staging test results as sufficient evidence of impact across all downstream systems.

C. Notify downstream system owners that a change is coming and schedule the rollout for the following week, without conducting a formal impact assessment.

D. Limit the rollout to systems that were explicitly included in staging testing, and defer all other downstream systems until a later release cycle.

✓ **Correct Answer: A. Assess the configuration impact on each downstream system before rolling out, and coordinate with downstream system owners as needed.**

Why: Assessing the configuration change's impact on each downstream system, and coordinating with their owners, is necessary because staging tests alone don't reveal how every dependent system will react to a prompt/version change. Relying on staging results alone (B), notifying without assessing (C), or arbitrarily limiting the rollout to already-tested systems (D) all leave real risk unaddressed.

Q059

SCENARIO · SINGLE CHOICE

You are choosing a Claude model for a high-volume classification task. Each classification is straightforward, latency requirements are tight, and per-request cost matters at scale. Which model would you choose?

A. A mid-tier Claude model selected by default, because mid-tier models balance quality and cost in a way the team can apply across most tasks.

B. Multiple Claude models in series, where each request runs through more than one model and the application

combines the outputs into a final classification.

✓ **C. A smaller, faster Claude model, because the task is straightforward and the workload prioritizes latency and per-request cost at scale.**

D. The largest, highest-capability Claude model, to maximize quality on every classification the application produces during normal operation across all requests.

✓ **Correct Answer: C. A smaller, faster Claude model, because the task is straightforward and the workload prioritizes latency and per-request cost at scale.**

Why: A smaller, faster model directly matches a workload that's straightforward, latency-sensitive, and cost-sensitive at scale — using the largest model (D) or defaulting to mid-tier (A) overspends on capability the task doesn't need, and chaining multiple models (B) adds cost and latency instead of reducing it.

Q060

SCENARIO • SINGLE CHOICE

Your Claude application is deployed to development, staging, and production environments. Each environment uses a different model version, different prompt versions, and different plugin dependencies, but the configuration is currently scattered across environment variables, hardcoded values, and undocumented setup scripts. How would you manage the configuration?

A. Use the latest available model version everywhere and stop pinning model versions, on the grounds that pinning adds maintenance overhead the team should aim to reduce.

B. Move all configuration into hardcoded application code to reduce reliance on external configuration sources that are difficult to track over time.

✓ **C. Consolidate the configuration into a version-controlled system documenting model version pinning, prompt versioning, and plugin dependencies for each environment.**

D. Standardize all environments to use the same configuration values to eliminate the differences between development, staging, and production.

✓ **Correct Answer: C. Consolidate the configuration into a version-controlled system documenting model version pinning, prompt versioning, and plugin dependencies for each environment.**

Why: Consolidating scattered configuration into a version-controlled system that documents model pinning, prompt versions, and plugin dependencies per environment brings order and traceability to what was previously undocumented and inconsistent. Removing pinning (A), hardcoding values (B), or forcing all environments identical (D) each remove flexibility the team actually needs.

Q061

SCENARIO • SINGLE CHOICE

You maintain a Claude application that uses Claude Sonnet 4.5 across several production workflows. Anthropic released Claude Sonnet 4.7, which your evaluation suite shows performing 8% better on your highest-volume task. However, this version produces different output formatting on two of your structured-extraction prompts that downstream consumers parse with regex-based code. To roll out

the upgrade, you would...

✓ **A. Adjust the prompts to constrain output format, re-run evaluations against the expected parsing-layer schema, then roll the model out with a feature flag and the option to revert per workflow.**

B. Switch the model identifier in the application's configuration to the new model, then monitor production for parsing failures to be fixed as failures surface.

C. Rewrite the downstream parsing code to accept a wider range of output formats to ensure the application is resilient to future model upgrades without requiring prompt changes.

D. Keep the older model in production to preserve the fragile parsing layer that may risk breaking with any model change.

✓ **Correct Answer: A. Adjust the prompts to constrain output format, re-run evaluations against the expected parsing-layer schema, then roll the model out with a feature flag and the option to revert per workflow.**

Why: Because the newer model changes output formatting that regex-based downstream parsers depend on, the safe rollout constrains the prompt to the expected format, re-validates against the parsing schema, and ships behind a feature flag with a per-workflow revert option. Just swapping the model ID and fixing issues as they surface (B), rewriting all downstream parsers (C), or staying on the old model indefinitely (D) are riskier or overly costly responses.

Q062

SCENARIO · SINGLE CHOICE

You are designing a Claude application that helps medical researchers analyze multi-step clinical case studies. The application must work through differential diagnoses by considering symptom patterns, weighing evidence across competing hypotheses, and showing intermediate reasoning steps before producing a final recommendation. The team is choosing among Claude's available model options. The model option best suited to this use case is...

A. Zero-shot prompting alone with no model option adjustments, which keeps the application's configuration as simple as possible.

✓ **B. Extended thinking, which lets the model reason through the differential diagnosis steps before producing the final recommendation.**

C. Fast mode, which prioritizes the lowest possible latency at the expense of reasoning depth on complex tasks.

D. A smaller model with a tighter context window, which encourages the model to focus its limited capacity on the task.

✓ **Correct Answer: B. Extended thinking, which lets the model reason through the differential diagnosis steps before producing the final recommendation.**

Why: Extended thinking lets the model work through the differential-diagnosis reasoning — weighing evidence, considering competing hypotheses — before producing a final recommendation, which fits a task that explicitly needs visible intermediate reasoning. Zero-shot with no adjustments (A), fast mode optimized for low latency (C), or a smaller model with a tighter window (D) all work against the reasoning depth this task requires.

Q063

SCENARIO · SINGLE CHOICE

Your Claude application's prompt was written months ago and has not been updated. The team has discovered through evals that the prompt produces good results on common cases but underperforms on a specific category of inputs that has grown in volume. How would you respond?

✓ **A. Iterate on the prompt to address the underperforming category, validate the change with evals, and continue refining as needed.**

B. Tell users to avoid the underperforming category by adding warnings in the application's user interface about handled inputs.

C. Replace the prompt with a new one aligned to the underperforming category, treating any common-case performance change as a known tradeoff.

D. Add the underperforming category to a separate Claude application with its own prompt so the original prompt does not change.

✓ **Correct Answer: A. Iterate on the prompt to address the underperforming category, validate the change with evals, and continue refining as needed.**

Why: Iterating on the prompt to target the underperforming category, and validating the change with evals, directly improves the weak spot while confirming it didn't regress the cases that already work well. Steering users away from the category (B), swapping the whole prompt and accepting a common-case regression (C), or forking a second application (D) are all more disruptive workarounds.

Q064

SCENARIO · SINGLE CHOICE

You are designing an agent that processes vendor invoices. The work involves a small number of well-understood steps, but occasionally an invoice arrives in an unexpected format that requires the system to decide between rerouting, requesting clarification, or flagging for human review. The most appropriate architecture for this system is...

A. A fully autonomous agent that handles every invoice from start to finish across all formats.

B. A manager agent that delegates each step of standard invoice processing to a dedicated subagent, with a separate subagent handling each unexpected format.

C. A single large prompt that processes every incoming invoice, both standard and unexpected, in one model call.

✓ **D. A workflow for the standard path with an agent invoked at the decision point for unexpected formats.**

✓ **Correct Answer: D. A workflow for the standard path with an agent invoked at the decision point for unexpected formats.**

Why: A workflow handles the well-understood standard steps deterministically, with an agent invoked only at the decision point where judgment is needed for unexpected formats — matching effort to where flexibility is actually required. A fully autonomous agent for everything (A), a manager delegating to a subagent per format (B), or one giant prompt for every case (C) either overuse agentic flexibility or underuse it.

Q065

SCENARIO · SINGLE CHOICE

You are building a Claude application that needs to maintain a persistent connection to a service that streams real-time updates. The team is unsure what communication pattern to use. Which communication pattern would you use?

- A. Repeated short-lived HTTP polling requests, where the application opens a new HTTP connection each time it checks for updates.
- ✓ B. A WebSocket, because WebSockets are designed for bidirectional, persistent, real-time communication between the client and the streaming service.
- C. A single long HTTP request the server holds open indefinitely, with no standard WebSocket framing on the connection.
- D. File-based communication where the service writes new updates to disk and the application polls the file system for changes.

✓ **Correct Answer: B. A WebSocket, because WebSockets are designed for bidirectional, persistent, real-time communication between the client and the streaming service.**

Why: WebSockets are purpose-built for persistent, bidirectional, real-time communication, which fits a long-lived connection to a service streaming real-time updates. Repeated HTTP polling (A), a single indefinitely-held HTTP request without WebSocket framing (C), or file-based polling (D) are all less suited to this pattern.

Q066

SCENARIO · SINGLE CHOICE

Your Claude agent has too many tools, and many of them have overlapping functionality. The agent often picks an inappropriate tool when several could plausibly handle a request. How would you address the tool selection problem?

- A. Add more tools to cover every variation of the requests the agent handles, on the grounds that more tools give the agent more accurate options to choose from.
- B. Remove all tools and rely on the agent's general capability instead, with the application losing the workflows that previously relied on tools.
- C. Add detailed examples to each tool's description so the agent can match incoming requests to the right tool by example, treating the examples as the team's selection mechanism.
- ✓ D. Restructure the tool set by consolidating overlapping tools, removing unused tools, and clarifying tool descriptions so each tool has a distinct purpose.

✓ **Correct Answer: D. Restructure the tool set by consolidating overlapping tools, removing unused tools, and clarifying tool descriptions so each tool has a distinct purpose.**

Why: Restructuring the tool set — consolidating overlapping tools, removing unused ones, and clarifying descriptions so each tool has a distinct purpose — directly fixes ambiguous tool selection. Adding even more tools (A) makes the ambiguity worse, removing tools entirely (B) loses needed functionality, and just adding examples to overlapping tools (C) treats a symptom rather than the underlying overlap.

Q067

SCENARIO · SINGLE CHOICE

You are designing a Claude application that will require structured JSON output for downstream processing. The output schema is well-defined, and downstream systems will reject malformed JSON.

- A. Structure the prompt to request output in a schema that is described in plain English, with downstream systems parsing whatever shape Claude produces.
- B. Define a clear schema and structure the prompt to request output in that schema, with downstream systems handling any validation needed.
- ✓ C. Define a clear schema, structure the prompt to request output in that schema, and validate Claude's output against the schema before passing it downstream.
- D. Avoid structured output and use free-form text everywhere instead, on the grounds that free-form text is more flexible and handles edge cases better than structured schemas.

✓ **Correct Answer: C. Define a clear schema, structure the prompt to request output in that schema, and validate Claude's output against the schema before passing it downstream.**

Why: For output a downstream system will reject if malformed, the full loop matters: define a clear schema, prompt for output in that schema, and then validate the actual output against it before passing it on. Skipping validation (A, B) or abandoning structure altogether (D) leaves the rejection risk in place.

Q068

SCENARIO · SINGLE CHOICE

You are setting up the configuration management approach for a new Claude Code project. Your team will use CLAUDE.md files and settings.json files to control behavior, and you want to make sure changes are tracked and reviewable. The configuration management approach would...

- A. Duplicate CLAUDE.md and settings.json files in multiple repositories to provide redundancy, on the grounds that a single source of truth is risky for project configuration.
- B. Version-control CLAUDE.md and settings.json files in a separate repository from the project's source code, so configuration evolves independently from the application code over time.
- C. Version-control CLAUDE.md alongside the project's source code and settings.json files in a separate repository from the project's source code.
- ✓ D. Version-control CLAUDE.md and settings.json files alongside the project's source code, with changes reviewed through standard pull request workflows the team applies.

✓ **Correct Answer: D. Version-control CLAUDE.md and settings.json files alongside the project's source code, with changes reviewed through standard pull request workflows the team applies.**

Why: Version-controlling both CLAUDE.md and settings.json alongside the source code, with changes going through standard pull request review, keeps configuration changes tracked and reviewable just like code. Duplicating files for 'redundancy' (A) or splitting configuration into a separate repo (B, C) fragments traceability instead of improving it.

Q069

SCENARIO · SINGLE CHOICE

Your team's Claude agent has accumulated several customizations that bypass the SDK's defaults, including custom history management, retry logic, and error handling. A new team member has proposed reverting all the customizations to maintain the codebase more easily. The tech lead disagrees and says each customization was added for a reason. How would you advise the team?

- A. Migrate the agent off the SDK and rebuild it with a custom loop.
- B. Revert all customizations to the SDK's defaults to standardize the codebase.
- C. Keep all customizations, trusting that the tech lead's original reasoning is still valid.

✓ D. Decide on each customization individually based on its original reason and the SDK's current capabilities.

✓ **Correct Answer: D. Decide on each customization individually based on its original reason and the SDK's current capabilities.**

Why: Deciding on each customization individually — weighing its original rationale against what the SDK's current capabilities actually offer — respects that some customizations may still be needed while others may now be redundant. Reverting everything (B) or keeping everything on faith (C) both skip that case-by-case evaluation, and abandoning the SDK entirely (A) is a disproportionate response.

Q070

SCENARIO · SINGLE CHOICE

You are designing an agent that handles a multi-step research task. You want the agent to break the task into smaller pieces, hand each piece to a focused subagent, and consolidate the results. The agent pattern you would apply is...

✓ A. An orchestrator and subagent pattern with specialized subagents assigned to each subtask.

- B. A memory pattern that stores the entire research history in advance, before any subtask begins execution.
- C. A context-window pruning pattern that drops each subtask's content after the agent moves on.
- D. A single tool-use loop that includes every tool the agent might need across all subtasks.

✓ **Correct Answer: A. An orchestrator and subagent pattern with specialized subagents assigned to each subtask.**

Why: An orchestrator/subagent pattern, where a top-level agent breaks the research task into pieces and hands each to a focused subagent before consolidating results, is exactly the decomposition-and-delegation structure described. A memory-only pattern (B), context pruning alone (C), or one flat tool-use loop covering every subtask (D) don't provide that decomposition.

Q071

SCENARIO · SINGLE CHOICE

You are deciding between deploying a Claude-powered agent on Anthropic's hosted infrastructure or self-hosting under a "bring your own cloud" model in your own AWS account. The agent processes customer data subject to your enterprise's data residency policies, but the team wants to ship quickly and avoid managing infrastructure. Which deployment model would you recommend?

A. Self-hosting under BYOC for an initial pilot, then evaluating whether to migrate to Anthropic-hosted infrastructure once the agent's data-handling patterns are better understood.

B. Deploying on Anthropic-hosted infrastructure while the team drafts a request to update the enterprise data residency policy to accommodate hosted AI deployments.

✓ C. Self-hosting under BYOC to satisfy the data residency requirement, while working with the infrastructure team to reduce the operational overhead of managing the deployment.

D. Deploying on Anthropic-hosted infrastructure to meet the team's shipping timeline, and flagging the data residency requirement for a follow-up compliance review after launch.

✓ **Correct Answer: C. Self-hosting under BYOC to satisfy the data residency requirement, while working with the infrastructure team to reduce the operational overhead of managing the deployment.**

Why: Self-hosting under BYOC satisfies the data residency requirement directly while the team works with infrastructure to reduce the operational burden — meeting the actual compliance constraint takes priority over shipping speed. Deploying on Anthropic-hosted infrastructure and hoping to fix policy later (B, D), or treating BYOC as a temporary pilot only (A), risk violating the residency requirement in the meantime.

Q072

SCENARIO · SINGLE CHOICE

A new agent your team built handles customer support tickets, but it routinely gets confused when a single ticket spans billing, shipping, and product issues. The agent often loses track of which sub-issue it has already addressed and revisits the same one. The team is considering architectural changes. What architectural change would you recommend?

A. Switch to a deterministic workflow that handles billing, shipping, and product issues in a fixed sequence.

B. Add detailed prompting that instructs the agent to track which sub-issues have been resolved and which remain.

✓ C. Introduce an orchestrator agent that delegates billing, shipping, and product sub-issues to dedicated subagents.

D. Increase the size of the agent's context window so it can hold the full ticket history at once.

✓ **Correct Answer: C. Introduce an orchestrator agent that delegates billing, shipping, and product sub-issues to dedicated subagents.**

Why: Introducing an orchestrator agent that delegates each sub-issue (billing, shipping, product) to a dedicated subagent gives each sub-issue its own focused tracking, which directly addresses losing track of what's been resolved. A fixed deterministic sequence (A), more detailed prompting alone (B), or simply enlarging the context window (D) don't give the same clean separation of concerns.

Q073

SCENARIO · SINGLE CHOICE

Your enterprise has a contract with AWS that requires Claude API calls to flow through Amazon Bedrock rather than the direct Anthropic API. Your team is building a new Claude application and is unfamiliar with this constraint. How would you build the application?

- A. Build two parallel implementations of every call, one for the direct Anthropic API and one for Bedrock, and pick the faster one at runtime.
- B. Build the application against the direct Anthropic API now and migrate to Bedrock in a follow-up release once the team has more experience with the Bedrock API.
- ✓ C. Configure the application to invoke Claude through the Bedrock-compatible API path while keeping the application's logic provider-agnostic.
- D. Build the application against the direct Anthropic API and ignore the contractual requirement to route Claude calls through Amazon Bedrock.

✓ **Correct Answer: C. Configure the application to invoke Claude through the Bedrock-compatible API path while keeping the application's logic provider-agnostic.**

Why: Configuring the application to call Claude through the Bedrock-compatible API path, while keeping the application logic provider-agnostic, satisfies the contractual requirement without over-coupling the code to Bedrock specifically. Building parallel implementations (A), delaying compliance to a follow-up release (B), or ignoring the contract (D) all leave the requirement unmet in the meantime.

Q074

SCENARIO · SINGLE CHOICE

A teammate is debugging a Claude application whose system prompt has grown to several hundred lines and now contains overlapping, contradictory, and obsolete instructions. How would you advise the teammate?

- A. Add more explicit instructions so the most recent rules dominate the model's interpretation of the prompt during each request.
- ✓ B. Audit the prompt for overlap, contradiction, and obsolete content, then refactor so each instruction is clear, current, and non-redundant.
- C. Tighten only the contradictory rules first, treating the overlap and obsolete content as lower-priority work the team can address later.
- D. Split the prompt across multiple system prompts so the model sees a smaller portion at any given time.

✓ **Correct Answer: B. Audit the prompt for overlap, contradiction, and obsolete content, then refactor so each instruction is clear, current, and non-redundant.**

Why: Auditing the prompt for overlap, contradiction, and obsolete content, then refactoring so every instruction is clear and current, is the only option that actually resolves a prompt that's become tangled over time. Adding more instructions on top (A), fixing only the contradictions (C), or splitting the prompt without addressing the redundancy (D) all leave the underlying mess in place.

Q075

SCENARIO · SINGLE CHOICE

A teammate has asked you to explain why your Claude agent's tools include detailed descriptions in the tool definition, even when the tool name is already descriptive. The teammate suggests removing

the descriptions to simplify the tool definitions. How would you respond?

A. Suggest replacing the descriptions with example calls embedded in the tool definition, treating example calls as a complete substitute for the prose description.

B. Agree with the teammate because tool names are sufficient for the model to choose the right tool on every request the agent handles.

✓ C. Explain that the model uses the tool description to decide when to call the tool, and descriptions disambiguate cases where the tool name is not enough.

D. Suggest moving the descriptions out of the tool definition and into a separate documentation file the team maintains so the tool definitions stay short and the descriptions remain available.

✓ **Correct Answer: C. Explain that the model uses the tool description to decide when to call the tool, and descriptions disambiguate cases where the tool name is not enough.**

Why: The model relies on tool descriptions to decide when a tool applies, and descriptions disambiguate cases where the name alone isn't specific enough — which is why they're worth keeping even when a name seems descriptive. Agreeing that names alone suffice (B), replacing descriptions with example calls only (A), or moving them out of the tool definition into separate docs (D) all reduce the information available to the model at decision time.

Q076

SCENARIO · SINGLE CHOICE

You are establishing the guardrail strategy for a Claude application. The team wants to ensure guardrail failure does not expose the application to unsafe behavior. The guardrail strategy would...

✓ A. Layer multiple guardrails so a single guardrail failure does not expose the application to unsafe behavior.

B. Apply guardrails at the application output level only and route flagged responses to a human reviewer before they are delivered to the user.

C. Implement a single comprehensive system prompt guardrail and validate its coverage against the application's full range of expected inputs.

D. Apply guardrails at the model level only and rely on the model's built-in safety behaviors to handle any cases the guardrail does not explicitly cover.

✓ **Correct Answer: A. Layer multiple guardrails so a single guardrail failure does not expose the application to unsafe behavior.**

Why: Layering multiple guardrails means a single guardrail failing doesn't leave the application exposed to unsafe behavior — defense in depth. Relying on a single comprehensive prompt guardrail (C), guardrails only at the output stage (B), or only at the model level (D) each create a single point of failure.

SECTION 5

Questions 77–95

Claude Certified Developer — Foundations · Practice Set

Q077

SCENARIO · SINGLE CHOICE

The Claude application your team built has grown over six months, and the prompt-handling code has accumulated duplication and tangled control flow. The functionality is working, but new features are getting harder to add. How would you address this?

✓ **A. Plan a refactoring pass to consolidate duplicated logic, separate concerns, and simplify control flow before adding new features.**

B. Refactor the prompt-handling code in small increments as part of each new feature ticket, treating the cleanup as a side effect of feature work.

C. Continue adding features and plan a refactoring pass after the next two release cycles when the team has more bandwidth for internal work.

D. Move all the prompt-handling code into a single large function to reduce the number of files developers have to navigate when reading the code.

✓ **Correct Answer: A. Plan a refactoring pass to consolidate duplicated logic, separate concerns, and simplify control flow before adding new features.**

Why: A dedicated refactoring pass to consolidate duplication, separate concerns, and simplify control flow addresses accumulated technical debt directly, rather than deferring it (C), squeezing it in as a side effect of unrelated feature tickets (B), or making the code harder to navigate by cramming it into one function (D).

Q078

SCENARIO · SINGLE CHOICE

A teammate has submitted a pull request that adds a Claude-powered feature to your service. The code works, but the prompt and model selection are hard-coded inline, error handling is missing, and there are no tests for the integration. What would you request during code review?

A. Approve the pull request and add the missing pieces yourself in a follow-up commit so the teammate can move on to other work immediately.

B. Approve the pull request as-is, on the grounds that the feature works in the happy path and the missing pieces can be added in follow-up commits.

C. Request changes that move prompt and model configuration to a configurable location and add tests, treating the

missing error handling as a follow-up release item.

✓ **D. Request changes that move prompt and model configuration to a configurable location, add error handling for Claude API failures, and add tests for the integration.**

✓ **Correct Answer: D. Request changes that move prompt and model configuration to a configurable location, add error handling for Claude API failures, and add tests for the integration.**

Why: A thorough code review should ask for configurable prompt/model settings, real error handling for Claude API failures, and integration tests — all in the same pass, since none of these are optional for production-quality code. Approving as-is (B), deferring the missing pieces to a follow-up (A, C), or letting error handling slide (C) all let known gaps ship.

Q079

SCENARIO • SINGLE CHOICE

Your team's Claude application has been in production for a year, and the team has decided to formalize its testing strategy. Currently, the team writes ad-hoc tests for individual features but has no overall testing approach. What testing approach would you formalize?

A. Adopt a test-driven development practice where unit tests are written before each feature is implemented and must pass before code is merged.

✓ **B. Define unit tests for individual functions, integration tests for the Claude integration, and end-to-end tests for critical user flows, applied consistently across the codebase.**

C. Continue writing ad-hoc tests as features ship and introduce a peer review step to ensure each test adequately covers the feature being released.

D. Define a single testing approach that uses end-to-end tests and apply it consistently across all new features as they are added to the codebase.

✓ **Correct Answer: B. Define unit tests for individual functions, integration tests for the Claude integration, and end-to-end tests for critical user flows, applied consistently across the codebase.**

Why: A formal testing strategy spans unit tests for individual functions, integration tests for the Claude integration itself, and end-to-end tests for critical user flows, applied consistently. Test-driven development as the sole practice (A), ad-hoc tests with peer review (C), or end-to-end tests alone (D) don't give the same layered coverage.

Q080

SCENARIO • SINGLE CHOICE

You have just shipped a new Claude-powered application to production. The development phase is complete, and the system is now in active use by internal teams. The next phase of work for this system is...

A. Retiring the system, since shipping is the final lifecycle stage for any application that reaches production.

B. Running a formal post-deployment review that assesses the development phase before the team does any further work on the system.

✓ **C. Operating and maintaining the system, including monitoring, responding to issues, and planning evolution.**

D. Handing the system over to a separate operations team that will manage it independently of the development team going forward.

✓ **Correct Answer: C. Operating and maintaining the system, including monitoring, responding to issues, and planning evolution.**

Why: Shipping to production starts the operate-and-maintain phase — monitoring, responding to issues, and planning further evolution — rather than ending the work. Treating shipping as the end state (A), handing off to a separate team with no continuity (D), or requiring a formal review before any further work happens (B) misrepresent what comes next.

Qo81

SCENARIO · SINGLE CHOICE

You are designing a Claude application that will process customer support tickets in two stages: a triage stage that classifies tickets and a response stage that drafts replies. The team is debating whether to use a single Claude call that handles both stages or separate Claude calls for each stage. How would you structure the application?

A. Use a single Claude call for triage and then use a non-Claude rule-based system for response generation, on the grounds that rule-based systems are more reliable for drafting replies.

B. Use multiple Claude calls in parallel that each draft a complete ticket reply, then have a fourth Claude call select the best one to send to the customer.

✓ **C. Use separate Claude calls for triage and response, because each stage has distinct inputs, outputs, and success criteria that benefit from focused prompts.**

D. Use a single Claude call for both stages, on the grounds that a single call is cheaper than multiple calls in any production Claude application setup.

✓ **Correct Answer: C. Use separate Claude calls for triage and response, because each stage has distinct inputs, outputs, and success criteria that benefit from focused prompts.**

Why: Separate Claude calls for triage and response fit the fact that each stage has distinct inputs, outputs, and success criteria, letting each prompt stay focused. Combining both stages into a single call (D), mixing in a non-Claude rule-based response step (A), or generating multiple parallel drafts with a fourth selection call (B) add complexity without matching the natural stage boundaries.

Qo82

SCENARIO · SINGLE CHOICE

The product team has described a new Claude feature in business terms: "agents should help our analysts produce client memos faster." You need to convert this into actionable technical requirements for the engineering team. Your first step would be to...

A. Ask the analysts about the current memo production process to see where they think Claude could be introduced as a prompt-driven drafting step.

B. Assess what similar agent-based features have been built internally or in the industry and use those precedents to scope the technical approach.

C. Examine what model capabilities and tier options are available and determine which best supports the memo drafting workflow described by the product team.

✓ D. Interpret the functional and infrastructure requirements implied by the business goal.

✓ **Correct Answer: D. Interpret the functional and infrastructure requirements implied by the business goal.**

Why: The first step is translating the business goal into the functional and infrastructure requirements it implies — the technical foundation everything else builds on. Asking analysts about their current process (A), surveying similar built features (B), or jumping straight to model/tier selection (C) are useful later but skip defining what's actually being built first.

Qo83

SCENARIO • SINGLE CHOICE

You are running Claude Code as part of an automated continuous integration pipeline. The pipeline needs Claude Code to execute a set of well-defined tasks without prompting for confirmation, and the output needs to be captured for downstream processing. How would you configure the pipeline?

A. Replace Claude Code with a different tool that does not require any configuration to operate without confirmation prompts in the pipeline.

✓ B. Run Claude Code in headless mode with the required permissions configured in settings.json and capture its output for downstream processing.

C. Disable Claude Code's confirmation prompts globally across all environments so the pipeline runs without interruption from any prompt.

D. Run Claude Code in interactive mode and have a developer manually approve every confirmation prompt while the pipeline executes its tasks.

✓ **Correct Answer: B. Run Claude Code in headless mode with the required permissions configured in settings.json and capture its output for downstream processing.**

Why: Running Claude Code in headless mode, with permissions configured in settings.json and output captured for downstream use, is exactly what a non-interactive CI pipeline needs. Swapping tools entirely (A), disabling confirmation prompts globally across all environments (C), or requiring manual approval in interactive mode (D) either overreach or defeat the purpose of automation.

Qo84

SCENARIO • SINGLE CHOICE

You are building a Claude application that processes 10,000 customer emails overnight to extract structured data. The work is non-interactive, runs once daily, and has a flexible completion window of several hours. Which Claude API would you use?

✓ A. The Batch API, which is designed for non-interactive workloads with flexible completion windows.

B. The streaming responses API to process each email and return partial results to a database as the model generates them.

C. The real-time Messages API, processing the emails one at a time sequentially to ensure consistent ordering of

results.

D. The real-time Messages API with concurrent requests to process the emails as fast as possible during the overnight window.

✓ **Correct Answer: A. The Batch API, which is designed for non-interactive workloads with flexible completion windows.**

Why: The Batch API is designed precisely for non-interactive workloads with a flexible completion window, which matches processing 10,000 emails overnight with no real-time requirement. Streaming (B) and the real-time Messages API, sequential or concurrent (C, D), are built for interactive or time-sensitive use cases this workload doesn't need.

Qo85

SCENARIO • SINGLE CHOICE

Your Claude application has multi-step workflows where each step's output is needed only briefly before the agent moves on. The cumulative tool output is filling the context window with content that is no longer relevant. How would you handle the accumulating tool output?

✓ **A. Apply tool output pruning to remove tool outputs that are no longer needed by later steps in the workflow.**

B. Apply prompt caching to the accumulated tool outputs so the application does not re-pay for the older content on each subsequent step.

C. Switch to a smaller Claude model that processes context more efficiently and treat any quality loss as a tradeoff for the cost reduction.

D. Keep every tool output in the context indefinitely so the agent has the full record of every step it has executed during the workflow.

✓ **Correct Answer: A. Apply tool output pruning to remove tool outputs that are no longer needed by later steps in the workflow.**

Why: Tool output pruning removes outputs that are no longer needed once later steps have moved past them, directly addressing context filling up with stale content. Prompt caching (B) reduces cost but doesn't remove volume, switching models (C) sidesteps the actual problem, and keeping everything indefinitely (D) is the behavior causing the issue.

Qo86

SCENARIO • SINGLE CHOICE

Your Claude application produces good responses for typical inputs but struggles with edge cases. You have several labeled examples of edge-case inputs and the desired response for each. You want to use these examples to improve the model's handling of edge cases. What is the best way to use these examples?

A. Embed the examples in a database for the model to find during inference.

✓ **B. Add the labeled edge-case examples to the prompt as few-shot examples so the model can learn the pattern.**

C. Train a custom model on the edge-case examples and deploy that custom model in place of the team's current Claude integration.

D. Tell users to avoid submitting the edge-case inputs to the application by adding warnings in the application's user interface.

✓ **Correct Answer: B. Add the labeled edge-case examples to the prompt as few-shot examples so the model can learn the pattern.**

Why: Adding the labeled edge-case examples to the prompt as few-shot examples lets the model learn the desired pattern directly at inference time. Embedding them in a database for the model to 'find' (A) isn't how in-context learning works, training a custom model (C) is a disproportionate response, and telling users to avoid the inputs (D) doesn't fix the underlying gap.

Qo87

SCENARIO · SINGLE CHOICE

You are integrating Claude into an application written in Python. The Claude SDK provides a Python client that wraps the underlying REST API. How would you integrate the SDK?

A. Call the REST API directly with raw HTTP requests so the application avoids the SDK's abstraction between the application code and the API.

✓ **B. Use the Claude Python SDK and let it handle authentication, retries, and response parsing through its standard documented patterns for Python integrations.**

C. Use a different LLM provider's SDK and translate the responses into Claude's API shape so the application can switch providers in the future.

D. Skip the SDK and embed Claude calls in shell commands invoked from Python, so that the application runs the calls outside the main Python process.

✓ **Correct Answer: B. Use the Claude Python SDK and let it handle authentication, retries, and response parsing through its standard documented patterns for Python integrations.**

Why: Using the official Claude Python SDK and letting it handle authentication, retries, and response parsing through its documented patterns is the straightforward, supported integration path. Calling the REST API directly (A), adopting a different provider's SDK and translating responses (C), or shelling out from Python (D) all add unnecessary complexity or fragility.

Qo88

SCENARIO · SINGLE CHOICE

A teammate has asked you to explain why the team's Claude application is billed for output tokens at a different rate than input tokens. They had assumed the rate was the same for both. How would you explain the difference?

A. Output tokens are typically billed at the same rate as input tokens, and the apparent rate difference is a billing error to report to Anthropic.

B. Output tokens are typically billed at a lower rate than input tokens, because output tokens are cheaper to produce than input tokens are to process.

C. Output tokens are not billed at all, because cost is determined entirely by the input tokens sent to the model on each request.

✓ **D. Output tokens are typically billed at a higher rate than input tokens, and cost models for the application**

should reflect both rates separately.

✓ **Correct Answer: D. Output tokens are typically billed at a higher rate than input tokens, and cost models for the application should reflect both rates separately.**

Why: Output tokens are typically billed at a different (generally higher) rate than input tokens because generating tokens is a different, more compute-intensive operation than processing them, and a cost model should reflect both rates separately. Assuming the rates should match and calling the difference an error (A), assuming output is free (C), or assuming output is always cheaper (B) all misdescribe how billing actually works.

Q089

SCENARIO • SINGLE CHOICE

You are designing a multi-step Claude workflow where some steps must reason without seeing the full prior conversation history. The team wants to keep specific context isolated to specific steps. The context engineering technique you would use is...

- A. Augmenting the context with all available content at every step so each step has access to the entire prior history of the workflow during its reasoning.
- B. Using a single global prompt that applies to every step in the workflow no matter what each step is reasoning about during its run.

✓ **C. Context isolation through subagents or multi-step agentic workflows that scope each step's context to only what the step needs.**

- D. Embedding the full prior history in each step regardless of whether the step needs the prior history for its reasoning.

✓ **Correct Answer: C. Context isolation through subagents or multi-step agentic workflows that scope each step's context to only what the step needs.**

Why: Context isolation — through subagents or multi-step workflows that scope each step's context to only what it needs — lets specific steps reason without seeing the full prior history, exactly as required. Feeding every step the full history regardless of need (A, D) or applying one global prompt to every step (B) work against that isolation.

Q090

SCENARIO • SINGLE CHOICE

Your Claude application requests structured JSON output from the model. Most of the time the JSON is well-formed, but occasionally Claude returns malformed JSON that breaks downstream processing. How would you handle the malformed output?

- A. Manually inspect every response before downstream processing so a human reviewer catches any malformed JSON before the application passes the response to downstream systems.

✓ **B. Add output validation that parses Claude's response against the expected schema and treats malformed output as a recognized error path with retry or fallback handling.**

- C. Switch to free-form text output so the application no longer depends on JSON parsing for any of the responses it sends to downstream systems during normal operation.

D. Retry the same request repeatedly until valid JSON appears in the model's response, with the retry loop adding delay to the application's response time on affected requests.

✓ **Correct Answer: B. Add output validation that parses Claude's response against the expected schema and treats malformed output as a recognized error path with retry or fallback handling.**

Why: Adding output validation that parses the response against the expected schema, with malformed output treated as a recognized error path with retry or fallback handling, is the resilient way to handle occasional malformed JSON. Manual inspection of every response (A) doesn't scale, abandoning structured output (C) loses its benefits, and blind retries (D) add latency without guaranteed success.

Q091

SCENARIO • SINGLE CHOICE

You are building a Claude application that needs to deliver model output to end users as it is generated, instead of waiting for the full response to complete. The Claude API mechanism you would use is...

A. Structured JSON output, which delivers responses only after the model has finalized the JSON shape across the entire response.

✓ **B. Streaming responses, which deliver tokens incrementally as the model generates them so users see output progressively.**

C. The Batch API, which delivers full responses after a delay suitable for non-interactive workloads.

D. Prompt caching, which speeds up the cost profile of future requests and does not affect the delivery timing of the first response.

✓ **Correct Answer: B. Streaming responses, which deliver tokens incrementally as the model generates them so users see output progressively.**

Why: Streaming responses deliver tokens incrementally as they're generated, which is exactly what's needed to show users output progressively instead of waiting for the full response. Structured JSON output (A) still waits for the full response, the Batch API (C) is for non-interactive delayed delivery, and prompt caching (D) affects cost, not delivery timing.

Q092

SCENARIO • SINGLE CHOICE

A teammate has asked you to explain when a Skill would be the right choice over an MCP server. The teammate is unsure how the two differ in practice when both can be reused across teams. How would you explain the distinction?

A. A Skill and an MCP server are equivalent extension mechanisms that the team can use interchangeably for any reusable capability that needs to be accessible across teams.

B. A Skill is the older mechanism and an MCP server is the newer one, so the team should prefer an MCP server for any reusable capability that the team builds going forward.

C. A Skill is preferable for cross-team reuse because it loads more efficiently than an MCP server during normal operation in the team's typical multi-team workloads.

✓ **D. A Skill bundles prompts, scripts, and data into a package the model loads as a unit while an MCP server exposes resources, tools, and prompts through a standard client interface.**

✓ **Correct Answer: D. A Skill bundles prompts, scripts, and data into a package the model loads as a unit while an MCP server exposes resources, tools, and prompts through a standard client interface.**

Why: A Skill bundles prompts, scripts, and data into a unit the model loads directly, while an MCP server exposes resources, tools, and prompts through a standard client interface — different mechanisms suited to different situations, not interchangeable (A) or one simply newer/better than the other (B, C).

Q093

SCENARIO · SINGLE CHOICE

You are starting a new Claude application and have a small set of well-labeled examples that demonstrate the desired output format. You want to use these examples to guide Claude's behavior. How would you guide the application's behavior?

✓ **A. Use multi-shot prompting by including the labeled examples in the prompt so Claude can match the desired output format on each request.**

B. Use zero-shot prompting and rely on the model's general capability to produce the desired output format consistently across all incoming requests.

C. Embed the examples in a database and retrieve them at runtime as reference material for the team.

D. Train a custom model on the labeled examples before deployment so that the application does not need to include the examples in any prompt during operation.

✓ **Correct Answer: A. Use multi-shot prompting by including the labeled examples in the prompt so Claude can match the desired output format on each request.**

Why: Multi-shot prompting — including the labeled examples directly in the prompt — lets Claude match the demonstrated output format on each request using in-context learning. Zero-shot prompting (B) doesn't leverage the examples at all, embedding them in a database (C) isn't how the model would use them without retrieval integration, and training a custom model (D) is unnecessary for this scale of guidance.

Q094

SCENARIO · SINGLE CHOICE

You are setting up a Claude application that requires API keys for several external services. What is the best way to store the keys?

A. Put the keys in the application's configuration file and check the configuration file into the team's repository alongside the rest of the source code.

✓ **B. Store the keys in a secrets manager or environment-specific configuration that is not checked into source code, and load them at runtime.**

C. Use a single shared key across all external services, so any developer working on the application can find the keys easily during development.

D. Email the keys to each developer as needed and have each developer paste the keys into their local environment

when they begin working on the application's code.

✓ **Correct Answer: B. Store the keys in a secrets manager or environment-specific configuration that is not checked into source code, and load them at runtime.**

Why: Storing keys in a secrets manager or environment-specific configuration that's never checked into source code, and loading them at runtime, is standard secure key management. Checking keys into the repo (A), sharing a single key across all developers (C), or emailing keys around (D) all expose credentials unnecessarily.

Q095

SCENARIO · SINGLE CHOICE

Your Claude application is producing inconsistent outputs for similar inputs, even when using the same model and prompt. You want to debug the issue systematically. Your debugging approach would...

A. Lower the model's temperature and re-run the inconsistent inputs to determine whether the parameter change reduces output variability across runs.

B. Retry every request that produces an unexpected output and log the results to identify whether repeated calls converge on a consistent response pattern.

C. Assume inconsistent outputs reflect normal LLM non-determinism and document the variability as an accepted characteristic of the application's behavior.

✓ **D. Capture full traces of input, system prompt, user messages, model output, and parameters, then analyze the differences between consistent and inconsistent runs.**

✓ **Correct Answer: D. Capture full traces of input, system prompt, user messages, model output, and parameters, then analyze the differences between consistent and inconsistent runs.**

Why: Systematic debugging means capturing full traces — input, system prompt, user messages, output, and parameters — and comparing consistent versus inconsistent runs to find the actual difference. Just lowering temperature and re-running (A), retrying and hoping for convergence (B), or writing off the variability as expected without investigating (C) all skip the actual root-cause analysis.